REVERSE ENGINEERING

Volume 2 — Approfondissement

NIVEAU AVANCÉ — PRÉREQUIS : VOLUME 1

Ghidra avancé : scripting Python/Java, structures, types custom

x64dbg avancé : automatisation, tracing, plugins essentiels

Reverse Engineering du .NET : dnSpy, deobfuscation, protections

Shellcode : extraction, analyse, émulation avec Unicorn

Identifier les algorithmes cryptographiques

VM Protectors et OLLVM : quand le décompilateur ne suffit plus

Malwares fileless et techniques avancées d'évasion

Automatiser l'analyse : CAPE, scripts Python, pipelines IOC

Développer ses propres outils RE

Threat Hunting avec YARA : règles avancées et yara-python

Table des matières

1. Ghidra avancé — Aller au-delà de l'analyse basique

1.1 Structures et types custom — donner du sens aux données
1.2 Le Decompiler Override — corriger Ghidra quand il se trompe
1.3 Scripting Ghidra en Python (via Jython)
1.4 Scripting Ghidra en Java
1.5 Plugins essentiels : GhidrAI, BinDiff, ret-sync
1.6 Comparer deux binaires : trouver les changements entre variantes
1.7 Analyser les vtables C++ dans Ghidra

2. x64dbg avancé — Automatiser et aller plus vite

2.1 Scripting x64dbg : le langage de script natif
2.2 Conditional et Trace Breakpoints
2.3 Logging automatique d'appels API
2.4 Plugins essentiels : xAnalyzer, ScyllaHide, x64dbgpy
2.5 Memory tracing : suivre une valeur en mémoire
2.6 Patcher des binaires et sauvegarder les patches
2.7 Attacher x64dbg à un processus déjà en cours (live malware)

3. Reverse Engineering .NET

3.1 Pourquoi le .NET RE est différent
3.2 dnSpy : le Ghidra du .NET
3.3 Modifier et recompiler un assembly .NET
3.4 Obfuscation .NET : ConfuserEx, Eazfuscator, SmartAssembly
3.5 Déobfusquer ConfuserEx : méthode manuelle et outils
3.6 .NET native AOT et les nouvelles complications
3.7 Cas pratique : analyser un RAT .NET (AsyncRAT-like)

4. Shellcode — Analyse et Émulation

4.1 Qu'est-ce qu'un shellcode et comment le trouver
4.2 Extraire un shellcode d'un PE ou d'un document
4.3 Analyser un shellcode dans Ghidra
4.4 Émuler un shellcode avec Unicorn Engine
4.5 Shellcode commun : stager Meterpreter, CobaltStrike Beacon
4.6 Identifier et extraire la configuration d'un Beacon
4.7 scDbg et autres émulateurs de shellcode

5. Identifier les algorithmes cryptographiques

5.1 Pourquoi identifier la crypto est crucial
5.2 Reconnaître RC4, XOR, ROT
5.3 Reconnaître AES (patterns de S-Box, MixColumns)
5.4 Reconnaître RSA et les grands entiers
5.5 Hashing : MD5, SHA1, SHA256 en assembleur
5.6 FLIRT signatures et FindCrypt dans Ghidra

5.7 Décrypter une config malware étape par étape

6. VM Protectors et Obfuscation Avancée

6.1 Code virtualization : comment VMProtect et Themida fonctionnent
6.2 OLLVM : obfuscation au niveau LLVM
6.3 Reconnaissance d'un binaire virtualisé
6.4 Stratégies d'analyse pour les protecteurs forts
6.5 Déobfuscation OLLVM avec Miasm et d'autres outils
6.6 Quand l'analyse dynamique est la seule option

7. Malwares Avancés et Techniques Modernes

7.1 Fileless malware : code en mémoire uniquement
7.2 Reflective DLL Loading : anatomie complète
7.3 BOF (Beacon Object Files) de CobaltStrike
7.4 Kernel-mode malware : survol des rootkits
7.5 Living off the Land Binaries (LOLBins)
7.6 WMI persistence et Event Subscription

8. Automatiser l'Analyse RE

8.1 CAPE Sandbox : déploiement et utilisation
8.2 Écrire un script d'extraction d'IOCs en Python
8.3 Pipeline d'analyse automatique
8.4 Intégration avec ton SIEM via API
8.5 Fuzzy hashing et clustering de samples

9. YARA Avancé et Threat Hunting

9.1 YARA en profondeur : modules, conditions complexes
9.2 Écrire des règles pour détecter les familles, pas les samples
9.3 yara-python : intégrer YARA dans tes scripts
9.4 Threat hunting proactif : scanner les process en mémoire
9.5 Maintenir une base de règles YARA efficace

10. Développer ses Propres Outils RE

10.1 Parser un PE en Python de zéro
10.2 Écrire un émulateur simple avec Unicorn
10.3 Hooker des APIs avec Frida pour l'analyse dynamique
10.4 Automatiser Ghidra en batch (headless)
10.5 Construire un extracteur de config générique

CHAPITRE 1

Ghidra Avancé

Maîtriser Ghidra au-delà des bases — structures, scripting, comparaison de binaires

1.1 Structures et types custom — donner du sens aux données

La majorité des malwares utilisent des structures C pour organiser leurs données : configurations, données de threads, état interne. Quand Ghidra désassemble un accès à {ic('[rbx+0x18]')}, il te dit qu'il y a un accès à 24 bytes après l'adresse dans RBX. Mais si tu sais que RBX pointe vers une structure de configuration malware, tu peux dire que c'est {ic('cfg->c2_port')}. La différence en termes de lisibilité du pseudocode est massive.

La première étape est toujours d'identifier qu'il s'agit d'une structure. Le signal : un même registre (ou une même variable locale) est déréférencé à plusieurs offsets différents dans la même fonction ou des fonctions liées. `[RBX+0]`, `[RBX+8]`, `[RBX+0x10]`, `[RBX+0x18]` — RBX contient un pointeur vers une structure et le code accède à ses différents champs.

Créer une structure dans Ghidra : méthode

Ouvre le Data Type Manager (Window → Data Type Manager). Clic droit sur ton projet → New → Structure. Donne un nom explicite ({ic('MalwareConfig')}, {ic('BotContext')}, etc.). Commence à ajouter des champs selon ce que tu observes dans le code.

Comment déterminer la taille et le type de chaque champ ? Observe comment Ghidra accède à l'offset. Si tu vois `MOVZX EAX, BYTE [RBX+5]` → le champ à offset 5 est un byte (BYTE, CHAR, ou un flag). Si tu vois `MOV EAX, DWORD [RBX+8]` → le champ à offset 8 est un DWORD (4 bytes, int ou BOOL). Si tu vois `MOV RAX, QWORD [RBX+0x10]` → 8 bytes, probablement un pointeur (LPWSTR, HANDLE, etc.). Le contexte d'utilisation (passé à quelle API, comparé à quoi) te dit le type sémantique.

Identifier et définir une structure malware

```
; Exemple : identifier une structure de config malware dans l'assembleur

; Dans la fonction 'init_bot', on voit :
MOV  RAX, [RBX + 0x00]    ; RAX = pointeur (adresse) → probablement char* ou wchar_t*
MOV  EDX, [RBX + 0x08]    ; EDX = DWORD → port ? version ? taille ?
MOVZX ECX, BYTE [RBX+0x0C]; ECX = BYTE → flag ? mode ?
MOV  RSI, [RBX + 0x10]    ; RSI = pointeur → autre string ? callback ?
MOV  EDI, [RBX + 0x18]    ; EDI = DWORD → délai de beacon ? sleep timer ?

; Contexte d'utilisation :
; RAX (offset 0) est passé à InternetConnectW comme lpServerName → c2_host
```

```
; EDX (offset 8) est passé comme nServerPort → c2_port
; ECX (offset 0xC) est comparé à 1 et 2 → protocol_type (HTTP=1, HTTPS=2)
; RSI (offset 0x10) est passé à RegOpenKeyExW → registry_key_path
; EDI (offset 0x18) est passé à Sleep() → beacon_interval_ms

; Structure Ghidra résultante :
; struct BotConfig {
;   wchar_t*  c2_host;          // +0x00 (8 bytes, pointeur)
;   uint32_t  c2_port;          // +0x08 (4 bytes)
;   uint8_t   protocol_type;    // +0x0C (1 byte)
;   uint8_t   _padding[3];      // +0x0D (3 bytes alignement)
;   wchar_t*  registry_path;    // +0x10 (8 bytes, pointeur)
;   uint32_t  beacon_interval;  // +0x18 (4 bytes, ms)
; };

; Dans Ghidra : clic droit sur la variable RBX → 'Retype Variable'
; Saisis le nom de ta structure : BotConfig *
; Le Decompiler régénère immédiatement le pseudocode avec les noms de champs !
```

Appliquer une structure Windows existante

Windows est livré avec des milliers de structures définies (PROCESS_INFORMATION, STARTUPINFOW, SECURITY_ATTRIBUTES, etc.). Ghidra inclut la plupart d'entre elles dans son Data Type Manager, sous le groupe {ic('windows_vs12_32.gdt')} ou {ic('windows_vs12_64.gdt')}. Si une variable locale est utilisée comme paramètre d'une WinAPI connue, Ghidra peut souvent appliquer automatiquement le bon type — mais parfois il faut le faire manuellement.

Appliquer les types Windows dans Ghidra

```
// Exemple : CreateProcess utilise STARTUPINFOW et PROCESS_INFORMATION

// Dans Ghidra, tu vois souvent ça AVANT d'avoir appliqué les types :
undefined  local_70[104];
undefined8 local_8;
CreateProcessW(exe_path, NULL, NULL, NULL, FALSE, 0,
               NULL, NULL, &local_70, &local_8);

// C'est bizarre : local_70 est un buffer de 104 bytes, local_8 est 8 bytes.
// STARTUPINFOW = 104 bytes (structure documentée MSDN)
// PROCESS_INFORMATION = 32 bytes (mais Ghidra ne montre que le premier champ ici)

// Retype local_70 en STARTUPINFOW, local_8 en PROCESS_INFORMATION
// Ghidra régénère :
STARTUPINFOW    startup_info;
```

```
PROCESS_INFORMATION proc_info;
memset(&startup_info, 0, sizeof(STARTUPINFOW));
startup_info.cb = sizeof(STARTUPINFOW);  // Champ obligatoire
CreateProcessW(exe_path, NULL, NULL, NULL, FALSE,
               CREATE_SUSPENDED,          // Flag visible maintenant !
               NULL, NULL, &startup_info, &proc_info);
// proc_info.hProcess est maintenant le handle du processus créé
// → C'est Process Hollowing ! CREATE_SUSPENDED confirmé.
```

1.2 Le Decompiler Override — corriger Ghidra quand il se trompe

Le décompilateur Ghidra est excellent mais il a ses limites. Les problèmes les plus courants : fonctions non reconnues (du code traité comme des données), mauvaise convention d'appel inférée (certains appels indirects ou appels de pointeurs de fonction), variables incorrectement unifiées (Ghidra pense que deux variables locales sont la même). Voici comment corriger ces problèmes.

Problème 1 : Code non reconnu comme fonction

Ça arrive souvent avec du shellcode injecté, des fonctions après un JMP inconditionnel (Ghidra arrête l'analyse aux sauts), ou du code dans des sections non-standard. Solution : place le curseur au début de la zone de code suspecte, appuie sur `D` pour désassembler (tu verras les instructions), puis appuie sur `F` pour créer une fonction. Ghidra analyse le flow et crée la fonction avec son prologue/épilogue.

Problème 2 : Mauvaise calling convention sur un appel indirect

Quand un malware appelle une fonction via un pointeur de fonction ({ic('CALL [RBX+0x20]')}), Ghidra ne connaît pas toujours la signature de la fonction cible. Il suppose souvent une convention par défaut qui peut être fausse. Solution : identifie d'abord la fonction pointée (xrefs, analyse du contexte), puis définis le type du pointeur de fonction avec la bonne signature.

Corriger les vtables C++ dans Ghidra

```
// Problème typique : vtable C++ mal interprétée

// Ghidra génère :
(**(code **)(*pObject + 0x28))(pObject, arg1, arg2);

// C'est un appel de méthode virtuelle C++
// pObject est un pointeur vers un objet C++
// *pObject = pointeur vers la vtable
// *(vtable + 0x28) = 5ème entrée de la vtable (0x28 / 8 = 5)

// Pour mieux comprendre :
```

```
// 1. Va à l'adresse de la vtable dans Ghidra
// 2. Crée un tableau de pointeurs (Data → Array of pointers)
// 3. Navigue dans chaque entrée pour voir quelle méthode c'est
// 4. Définis les types de fonctions dans le Data Type Manager
// 5. Applique un type de structure 'vtable' au premier champ de pObject

// Après correction :
pObject->vtable->Encrypt(pObject, buffer, size);
// Beaucoup plus lisible !
```

1.3 Scripting Ghidra en Python (via Jython)

Ghidra embarque Jython (Python 2.7 via JVM). C'est daté (Python 2.7 en 2025...) mais fonctionnel. La bonne nouvelle : une vraie intégration Python 3 arrive progressivement via PyGhidra. Les scripts Ghidra te permettent d'automatiser des tâches répétitives : renommer toutes les fonctions qui correspondent à un pattern, extraire tous les strings déchiffrés, analyser les xrefs d'une API sur des milliers de fonctions en quelques secondes.

Pour écrire un script : Window → Script Manager → New Script → Python. L'éditeur de scripts s'ouvre avec un template de base. Tu as accès à toutes les APIs Ghidra via l'objet `currentProgram` et les helpers comme `getFunctionAt()` ou `getInstructionAt()`.

Script Ghidra Python — Traquer les imports dynamiques

```
# Script Ghidra Python — Trouver tous les appels à GetProcAddress
# et logger le nom de la fonction dynamiquement importée

# Dans Ghidra : Window → Script Manager → New Script

# @author SOC Analyst
# @category Malware Analysis
# @keybinding

from ghidra.app.script import GhidraScript
from ghidra.program.model.symbol import RefType

# getReferencesTo est disponible via currentProgram
listing = currentProgram.getListing()
symbolTable = currentProgram.getSymbolTable()

# Trouver le symbole GetProcAddress
gpa_symbols = list(symbolTable.getSymbols('GetProcAddress'))
if not gpa_symbols:
    print('GetProcAddress non trouvé dans les imports')
    raise Exception('stop')
```

```
gpa_addr = gpa_symbols[0].getAddress()
print(f'GetProcAddress à : {gpa_addr}')

# Trouver toutes les références vers GetProcAddress
ref_mgr = currentProgram.getReferenceManager()
refs = ref_mgr.getReferencesTo(gpa_addr)

results = []
for ref in refs:
    call_addr = ref.getFromAddress()
    # Remonter de quelques instructions pour trouver le 2ème argument
    # (RDX en x64 = nom de la fonction)
    instr = listing.getInstructionAt(call_addr)
    # Chercher le MOV RDX, [addr_string] juste avant
    check_instr = instr
    for _ in range(8):  # 8 instructions avant le CALL
        check_instr = check_instr.getPrevious()
        if check_instr is None:
            break
        mnemonic = check_instr.getMnemonicString()
        # Si on trouve LEA RDX ou MOV RDX → c'est l'argument du nom
        if mnemonic in ['LEA', 'MOV'] and 'RDX' in str(check_instr.getDefaultOperandRep
resentation(0)):
            # Essayer de lire la string à l'adresse référencée
            ops = check_instr.getDefaultOperandRepresentation(1)
            results.append(f'@ {call_addr} : GetProcAddress({ops})')
            break

for r in results:
    print(r)

# Output typique :
# @ 0x140003a20 : GetProcAddress([addr:rdata_VirtualAllocEx])
# @ 0x140003a50 : GetProcAddress([addr:rdata_WriteProcessMemory])
# → Tu vois immédiatement les APIs dynamiquement importées !
```

Script : extraire et déchiffrer les XOR strings automatiquement

Un pattern encore plus puissant : écrire un script qui trouve les boucles XOR, extrait les données et la clé, et te donne les strings déchiffrées directement dans la console Ghidra.

Script Ghidra — Extractor de strings XOR

```
# Script Ghidra Python — Extraire et déchiffrer les strings XOR
# Suppose que le pattern est : boucle avec XOR byte [ptr], key

from ghidra.program.model.mem import MemoryAccessException

memory = currentProgram.getMemory()
listing = currentProgram.getListing()

def read_bytes(addr, length):
    result = []
    for i in range(length):
        try:
            result.append(memory.getByte(addr.add(i)) & 0xFF)
        except:
            break
    return result

def xor_decrypt(data, key):
    return ''.join(chr(b ^ key) if 32 <= (b^key) <= 126 else '.' for b in data)

# Chercher toutes les instructions XOR avec une constante immédiate
# Pattern : XOR byte ptr [reg], imm8
found_strings = []

instructions = listing.getInstructions(True)
for instr in instructions:
    if instr.getMnemonicString() != 'XOR':
        continue
    # Vérifier si c'est XOR [mem], imm
    ops = str(instr)
    if 'byte ptr' in ops.lower() and any(c.isdigit() for c in ops.split(',')[-1]):
        # Extraire la clé XOR (dernier opérande)
        try:
            key_str = ops.split(',')[-1].strip()
            key = int(key_str, 16) if key_str.startswith('0x') else int(key_str)
            if 1 <= key <= 255:
                found_strings.append((instr.getAddress(), key))
        except:
            pass

print(f'Trouvé {len(found_strings)} sites de déchiffrement XOR potentiels')

# Pour chaque site, essayer de trouver le buffer source
```

```
# (simplifié - en pratique il faudrait tracer le pointeur en amont)
for addr, key in found_strings[:10]:  # Limiter à 10 pour l'exemple
    print(f'XOR key=0x{key:02X} @ {addr}')
```

1.4 Scripting Ghidra en Java

Pour des scripts plus complexes ou des plugins permanents, Java est la voie. L'API Ghidra est Java-native, donc tu as accès à toutes les fonctionnalités sans les limitations de Jython. Voici les classes les plus utiles à connaître :

Script Ghidra Java — Renommage automatique

```
// Script Ghidra Java — Renommer les fonctions selon leur comportement
// Exemple : toute fonction qui appelle UNIQUEMENT des fonctions de chiffrement
// est renommée 'crypto_helper_XXXX'

import ghidra.app.script.GhidraScript;
import ghidra.program.model.listing.*;
import ghidra.program.model.symbol.*;
import ghidra.program.model.address.*;
import java.util.*;

public class RenameCryptoFunctions extends GhidraScript {

    // APIs crypto Windows que les malwares utilisent
    private static final Set<String> CRYPTO_APIS = new HashSet<>(Arrays.asList(
        "BCryptEncrypt", "BCryptDecrypt", "CryptEncrypt", "CryptDecrypt",
        "BCryptOpenAlgorithmProvider", "BCryptGenerateSymmetricKey",
        "BCryptCreateHash", "BCryptHashData"
    ));

    @Override
    public void run() throws Exception {
        FunctionManager funcMgr = currentProgram.getFunctionManager();
        FunctionIterator functions = funcMgr.getFunctions(true);
        int renamed = 0;

        while (functions.hasNext()) {
            Function func = functions.next();
            if (func.getName().startsWith("FUN_")) {  // Seulement les non-nommées
                if (callsCryptoAPI(func)) {
                    String newName = "crypto_func_" + func.getEntryPoint();
                    func.setName(newName, SourceType.USER_DEFINED);
```

```
                    println("Renamed: " + func.getEntryPoint() + " → " + newName);
                    renamed++;
                }
            }
        }
        println("Total renommé : " + renamed);
    }

    private boolean callsCryptoAPI(Function func) {
        // Cherche les références sortantes vers les APIs crypto
        for (Reference ref : func.getCallFixups()) {
            String callee = ref.getToAddress().toString();
            if (CRYPTO_APIS.contains(callee)) return true;
        }
        return false;
    }
}
```

1.5 Plugins essentiels : BinDiff et ret-sync

BinDiff — Comparer deux binaires

BinDiff est l'outil de référence pour comparer deux binaires. Il te permet de trouver les différences entre deux versions d'un malware, ou de trouver les similarités entre un malware inconnu et une famille connue. Par exemple : tu as un sample inconnu et un sample RedLine v4 connu. BinDiff te dit quelles fonctions sont identiques (100% match), lesquelles ont été modifiées (90% match) et lesquelles sont nouvelles. En quelques minutes, tu sais si c'est une variante de RedLine ou un nouveau malware.

Installation : télécharge BinDiff (Google, gratuit). Il s'intègre directement dans Ghidra comme plugin. Workflow : importe les deux binaires dans le même projet Ghidra, analyse les deux, puis File → Export → BinDiff sur chacun. Lance BinDiff sur les deux exports et visualise les différences.

ret-sync — Synchroniser Ghidra et x64dbg

ret-sync est un plugin qui synchronise la position entre ton désassembleur (Ghidra) et ton debugger (x64dbg, WinDbg). Quand x64dbg s'arrête sur un breakpoint, Ghidra se positionne automatiquement sur l'instruction correspondante. Tu navigues dans x64dbg et tu vois en temps réel le pseudocode C dans Ghidra. C'est un gain de productivité énorme : tu n'as plus à chercher l'adresse dans Ghidra.

ret-sync — Installation et utilisation

```
# Installation ret-sync

# 1. Côté Ghidra : installer le plugin
```

```
# Télécharger ret-sync depuis https://github.com/bootleg/ret-sync
# Copier retsync_ghidra.jar dans le répertoire extensions de Ghidra
# Ghidra → File → Install Extensions → Activer retsync

# 2. Côté x64dbg : installer le plugin
# Copier retsync.dp64 dans x64dbg/plugins/

# 3. Utilisation :
# Lancer Ghidra avec le projet ouvert
# Activer ret-sync dans Ghidra : Window → retsync → Start listening
# Lancer le malware dans x64dbg
# Dans x64dbg : commande '!sync' pour activer la sync

# Résultat :
# Chaque pas dans x64dbg → Ghidra se positionne automatiquement
# Tu vois le pseudocode C en temps réel pendant l'exécution
# Double productivité : dynamique + statique simultanément
```

1.6 Comparer des variantes : workflow

Quand tu analyses une nouvelle variante d'une famille connue, tu n'as pas besoin de tout réanalyser. Voici le workflow pour aller vite :

1. BinDiff les deux versions. Identifie les fonctions identiques (tu peux ignorer le core), les fonctions modifiées (à analyser en détail), et les nouvelles fonctions (nouvelles capacités potentielles).

2. Importe les annotations. BinDiff peut importer les noms et commentaires de la version connue vers la nouvelle. Tu récupères instantanément tout le travail d'analyse précédent.

3. Focalise sur les diff. Analyse seulement les fonctions modifiées et nouvelles. Sur un malware de 500 fonctions, si 480 sont identiques, tu analyses 20 fonctions. Une heure de travail au lieu d'une journée.

CHAPITRE 2

x64dbg Avancé

Automatiser le debugging, tracer l'exécution, patches permanents

2.1 Le langage de script x64dbg

x64dbg dispose d'un langage de script natif qui te permet d'automatiser des séquences de debugging. C'est particulièrement utile pour : logger automatiquement les arguments de toutes les APIs appelées, bypasser des anti-debugs complexes en une commande, extraire des données de la mémoire à des moments précis.

Les scripts s'exécutent depuis la console x64dbg ou depuis le gestionnaire de scripts. La syntaxe est simple : des commandes x64dbg sur chaque ligne, avec des variables ({ic('$var')}), des conditions ({ic('jnz')}), et des boucles ({ic('loop')}).

x64dbg Scripting — Logger des APIs automatiquement

```
; Script x64dbg — Logger tous les appels CreateFileW avec les arguments
; Ce script pose un BP sur CreateFileW et loggue le nom du fichier
; à chaque appel avant de continuer l'exécution automatiquement

; Définir le breakpoint sur CreateFileW
bp CreateFileW

; Définir un handler automatique sur ce BP
; (Accessible via l'onglet Breakpoints → clic droit → Edit Commands)

; === Commandes à exécuter quand le BP CreateFileW se déclenche ===
; RCX = lpFileName (pointeur vers wchar_t)
; Lire la string Unicode à [RCX]
log "CreateFileW: {w:[rcx]}"  ; {w:[rcx]} = string Unicode à l'adresse RCX

; Continuer l'exécution sans interaction manuelle
run

; === Autres exemples de format de log ===
log "VirtualAlloc size={rdx} protect={r9}"    ; Afficher des valeurs de registres
log "RegSetValue key=[rdx->wstring]"

; Pour un log dans un fichier (trace permanente)
logfile "C:\\analysis_log.txt"
log "[{cip}] CreateFileW: {w:[rcx]}"    ; cip = adresse de l'instruction courante
```

2.2 Trace Logging — Enregistrer toutes les instructions

Le trace logging enregistre chaque instruction exécutée dans un log. C'est l'équivalent d'un enregistrement complet de l'exécution. C'est puissant mais ça génère énormément de données — utilise-le sur une plage d'exécution limitée, pas sur tout le programme.

Le cas d'usage typique : tu suspectes une boucle de déchiffrement dans une fonction, mais tu n'arrives pas à la comprendre statiquement. Tu actives le trace juste avant l'entrée dans cette fonction, tu le désactives à la sortie. Le log te montre chaque instruction avec les valeurs des registres — tu peux suivre la transformation des données instruction par instruction.

Trace Logging — Enregistrer une boucle de déchiffrement

```
; Activer le trace logging dans x64dbg

; Méthode 1 : Menu Debug → Trace Into / Trace Over
; → Exécute instruction par instruction ET enregistre dans le trace log

; Méthode 2 : Commandes pour trace conditionnel
traceintoconditional 1000, "eip=0x140003000"
; → Trace jusqu'à 1000 instructions ou jusqu'à arriver à 0x140003000

; Méthode 3 : trace avec log format custom
tracelog "xor_loop.txt", "[{cip}] {dis:cip} | RAX={rax} RCX={rcx} RSI={rsi}"
traceintoconditional 10000, "cip == 0x14000310A"  ; Fin de la boucle

; Résultat dans xor_loop.txt :
; [0x140003100] MOV AL, [RSI+RCX] | RAX=0x00 RCX=0x00 RSI=0x14004000
; [0x140003103] XOR AL, 0x42      | RAX=0xAB RCX=0x00 RSI=0x14004000
; [0x140003105] MOV [RSI+RCX], AL | RAX=0xE9 RCX=0x00 RSI=0x14004000
; [0x140003108] INC RCX           | RAX=0xE9 RCX=0x01 RSI=0x14004000
; [0x14000310A] CMP RCX, 0x40     | ...

; → Tu vois exactement : données chiffrées (0xAB), clé (0x42), résultat (0xE9)
; Et tu peux reconstruire la string complète offline avec Python !
```

2.3 Plugins essentiels

xAnalyzer — Annotations automatiques

xAnalyzer analyse les appels d'API et ajoute automatiquement des commentaires dans la vue assembleur, en montrant le nom et les valeurs des arguments pour chaque CALL vers une API Windows connue. C'est comme avoir un annotateur humain qui travaille en temps réel. Installation : copie xAnalyzer.dp64 dans le répertoire plugins de x64dbg.

x64dbgpy — Python dans x64dbg

x64dbgpy intègre Python 3 directement dans x64dbg. Tu peux écrire des scripts Python qui interagissent avec le debugger : lire et écrire de la mémoire, poser des breakpoints, lire les registres. C'est beaucoup plus puissant que le scripting natif x64dbg pour les analyses complexes.

x64dbgpy — Hook Python de VirtualAllocEx

```
# Script Python dans x64dbg via x64dbgpy

import x64dbgpy.pluginsdk as sdk
from x64dbgpy.pluginsdk import *

def hook_virtualalloc():
    """Intercepte VirtualAllocEx et affiche les arguments"""
    # Lire les arguments Windows x64 ABI
    hProcess  = Register.GetRAX()  # En fait RCX pour 1er arg
    # On est DANS VirtualAllocEx, donc les args sont dans RCX, RDX, R8, R9
    hProcess  = Registers.RCX
    lpAddress = Registers.RDX
    dwSize    = Registers.R8
    flAllocT  = Registers.R9

    # Lire flProtect depuis la stack (5ème arg)
    rsp_val   = Registers.RSP
    flProtect = Memory.ReadDword(rsp_val + 0x28)  # Shadow space + offset

    protect_names = {
        0x10: 'PAGE_EXECUTE',
        0x20: 'PAGE_EXECUTE_READ',
        0x40: 'PAGE_EXECUTE_READWRITE',  # ← TRÈS SUSPECT
        0x04: 'PAGE_READWRITE',
    }
    protect_str = protect_names.get(flProtect, f'0x{flProtect:02X}')

    print(f'[VirtualAllocEx] pid={hProcess} size=0x{dwSize:X} protect={protect_str}')
    if flProtect == 0x40:
        print('  ↑ ALERTE : PAGE_EXECUTE_READWRITE = probable injection de shellcode !'
)

# Poser le hook
Breakpoint.Add(Debugger.GetProcAddress('kernel32.dll', 'VirtualAllocEx'),
               hook_virtualalloc)

print('Hook VirtualAllocEx posé. Run le programme.')
```

2.4 Attacher x64dbg à un processus en cours

Dans certaines situations, tu ne peux pas lancer le malware depuis x64dbg directement. Par exemple, si le malware est lancé par un document Word (macro), ou s'il est lancé par un autre processus parent (dropper qui lance payload). Dans ce cas, tu lances le processus normalement puis tu attaches x64dbg à lui.

Méthode : File → Attach → sélectionne le processus dans la liste. Attention : quand tu attaches, le processus est mis en pause. Tu dois d'abord poser tes breakpoints AVANT d'appuyer sur F9 pour le reprendre. Les breakpoints que tu veux : les APIs suspectes que tu as identifiées en analyse statique. Une fois les BPs posés → F9 → le processus reprend et s'arrêtera à tes breakpoints.

Cas particulier : le malware vérifie s'il est attaché avec CheckRemoteDebuggerPresent. Dès l'attach, ScyllaHide doit être activé pour bypasser ces checks. Dans les options x64dbg : Options → ScyllaHide → cocher 'Anti Debug Active' avant d'attacher.

2.5 Patcher un binaire et sauvegarder

Parfois la solution la plus rapide est de patcher le binaire directement : bypasser un check anti-debug, forcer un chemin d'exécution, NOPer une routine malveillante. x64dbg te permet de patcher en mémoire (temporaire, durée de la session) ou de sauvegarder les patches dans un nouveau fichier EXE (permanent).

Patcher des binaires dans x64dbg

```
; Patcher dans x64dbg — exemples pratiques

; Cas 1 : Bypasser IsDebuggerPresent
; Le code original :
call IsDebuggerPresent    ; Retourne 1 si debuggé dans EAX
test EAX, EAX
jnz  exit_if_debugger     ; Si EAX != 0 → quitte

; Patch : MOV EAX, 0 avant le TEST (ou NOP le JNZ)
; Dans x64dbg : double-clic sur l'instruction → modifie les bytes
; Ou : barre d'espace sur l'instruction → éditeur d'assembleur
; NOP (no operation) = 0x90 en x86

; Cas 2 : Forcer un chemin d'exécution
; Le code :
cmp EAX, 1
je  legitimate_path
call malicious_code       ; On veut éviter ça

; Patch le JE en JMP (saut inconditionnel)
; JE = 0x74 xx  →  JMP short = 0xEB xx (même offset)
```

```
; Cas 3 : Sauvegarder les patches dans un EXE
; Après avoir patché en mémoire, tout est perdu à la fermeture
; Pour sauvegarder :
; 1. File → Patch File → Save As → Choisir un nom pour le patched exe
; 2. OU utiliser le plugin Patcher : affiche tous les patches appliqués
;    avec option d'export vers un EXE modifié
```

CHAPITRE 3

Reverse Engineering .NET

dnSpy, déobfuscation ConfuserEx, et analyse de malwares .NET modernes

3.1 Pourquoi le .NET RE est fondamentalement différent

Les programmes .NET (C#, VB.NET, F#) ne sont pas compilés en code machine directement. Ils sont compilés en CIL (Common Intermediate Language), un bytecode qui est ensuite exécuté par le CLR (Common Language Runtime). Ce CIL est beaucoup plus proche du code source original que l'assembleur x64 d'un programme C++ compilé. Résultat : la décompilation .NET est presque parfaite. Tu récupères souvent du code C# quasi-identique à l'original.

Les malwares .NET sont très courants dans l'écosystème cybercriminel. AsyncRAT, QuasarRAT, AgentTesla, FormBook, Warzone, dcRAT — tous sont en .NET ou ont des variantes .NET. Pourquoi ? Parce que C# est plus facile à développer que C++, les binaires sont plus faciles à obfusquer (de nombreux obfuscateurs), et le CLR donne accès facile à toutes les APIs Windows.

La mauvaise nouvelle : les auteurs de malwares savent que la décompilation .NET est facile. Donc ils utilisent des obfuscateurs. ConfuserEx, Eazfuscator, SmartAssembly, .NET Reactor — ces outils rendent le code décompilé illisible sans déobfuscation préalable. On verra ça dans la section 3.4.

3.2 dnSpy : le Ghidra du .NET

dnSpy est le décompilateur .NET de référence. Il décompile les assemblies .NET en C# lisible, permet de naviguer dans les classes, méthodes, et propriétés, et — fonctionnalité unique — permet de modifier le code source, recompiler, et relancer directement. C'est comme si tu pouvais modifier et recompiler les malwares que tu analyses.

Ouvre un EXE .NET dans dnSpy. Tu vois immédiatement l'arborescence des namespaces, classes, et méthodes. Navigue dans une classe, sélectionne une méthode — le panneau de droite affiche le code C# décompilé. La qualité est souvent excellente sur du code non-obfusqué : noms de variables intelligibles (parfois les vrais noms si le debug info est présent), structures de contrôle correctement reconstruites.

Interface dnSpy — ce qui compte

Assembly Explorer : arborescence à gauche. Les assemblies .NET sont organisées en namespaces → classes → méthodes. Cherche la classe principale (souvent nommée comme le projet). La méthode {ic('Main()')} est toujours le point d'entrée.

Recherche : Edit → Search Assemblies (Ctrl+Shift+K). Tu peux chercher des strings, des noms de méthodes, ou des types. Cherche {ic('cmd')} ou {ic('powershell')} pour trouver les appels shell directement.

Debugger intégré : dnSpy inclut un debugger .NET. Tu peux mettre des breakpoints sur des lignes de code C# décompilé. C'est encore plus puissant qu'x64dbg pour le .NET : tu debug à un niveau plus haut, sur des objets C# typés.

dnSpy — Code .NET décompilé annoté

```
// Exemple : Code C# décompilé d'un stealer .NET dans dnSpy
// (AsyncRAT-like, non obfusqué pour cet exemple)

namespace Client.Helper
{
    public class KeyLogger
    {
        private static string _logBuffer = string.Empty;
        private const int WH_KEYBOARD_LL = 13;

        public static void Start()
        {
            // Installe le hook clavier bas niveau
            KeyLogger._hookID = NativeMethods.SetWindowsHookEx(
                13,  // WH_KEYBOARD_LL
                new NativeMethods.LowLevelKeyboardProc(KeyLogger.HookCallback),
                NativeMethods.GetModuleHandle(null),
                0u
            );
        }

        private static IntPtr HookCallback(int nCode, IntPtr wParam, IntPtr lParam)
        {
            if (nCode >= 0 && wParam == (IntPtr)256)  // WM_KEYDOWN
            {
                int vkCode = Marshal.ReadInt32(lParam);
                KeyLogger._logBuffer += ((Keys)vkCode).ToString();

                // Flush au C2 toutes les 100 frappes
                if (KeyLogger._logBuffer.Length > 100)
                {
                    Connection.Send(new[]
                    {
                        new string[] { "KeyLog", KeyLogger._logBuffer }
                    });
                    KeyLogger._logBuffer = string.Empty;
```

```
                }
            }
            return NativeMethods.CallNextHookEx(...);
        }
    }
}
// → En quelques secondes, tu as compris :
// 1. C'est un keylogger Windows hook (SetWindowsHookEx WH_KEYBOARD_LL)
// 2. Il bufferise 100 frappes avant d'envoyer
// 3. Il envoie via la classe Connection.Send (→ navigue dans cette classe pour le C2)
```

3.3 Modifier et recompiler un assembly .NET

C'est la fonctionnalité qui n'a pas d'équivalent dans le RE natif : tu peux modifier le code source d'un malware .NET et le recompiler pour comprendre son comportement, ou pour créer une version 'désarmée'.

Cas d'usage 1 : bypass d'anti-analyse. Le malware vérifie {ic('Environment.MachineName')} et quitte si c'est une sandbox. Dans dnSpy : double-clic sur la méthode → Edit Method → change la ligne de vérification par `return false;` → Compile. Maintenant le malware pense qu'il n'est pas dans une sandbox.

Cas d'usage 2 : rendre le malware 'verbeux'. Ajoute des {ic('Console.WriteLine()')} dans les méthodes intéressantes pour logger exactement ce qui se passe sans avoir à poser des breakpoints partout.

Cas d'usage 3 : extraire une config chiffrée. Si la config est déchiffrée dans une méthode {ic('DecryptConfig()')} et que le résultat est utilisé en interne sans être loggé, tu modifies la méthode pour qu'elle affiche le résultat dans une MessageBox ou un fichier.

3.4 Obfuscation .NET — ConfuserEx en profondeur

ConfuserEx est l'obfuscateur .NET open source le plus utilisé par les malwares. Il applique plusieurs transformations :

Renommage (Renaming) : tous les noms de classes, méthodes et variables sont remplacés par des caractères non-ASCII ou des séquences sans sens. Au lieu de {ic('class KeyLogger')} tu vois {ic('class \u0001\u0002\u0003')} ou {ic('class AAAA')}. C'est la première chose que tu vois et la moins gênante : tu ne peux pas te fier aux noms mais le code reste analysable.

Control Flow Obfuscation : le flux de contrôle est rendu non-linéaire. Les {ic('if/else/for')} normaux sont transformés en une machine à états avec des switchers opaques. Au lieu de {ic('if (x > 5) { A(); } else { B(); }')} tu vois un tableau de délégués, des indices calculés dynamiquement, et des sauts indirects. C'est la transformation la plus pénible à bypasser manuellement.

String Encryption : les strings sont chiffrées et déchiffrées au runtime. Au lieu de {ic('string url = "http://evil.ru";')} tu vois {ic('string url = Decrypt(0xA3F2);')} où Decrypt est une méthode obfusquée. FLOSS ne fonctionne pas directement sur le .NET — il faut déchiffrer les strings différemment.

Déobfusquer ConfuserEx : les outils

de4dot est l'outil de référence pour la déobfuscation .NET. Il supporte une trentaine d'obfuscateurs différents, dont ConfuserEx, Eazfuscator, etc. Il nettoie le renommage, essaie de restaurer le flow control, et déchiffre les strings quand c'est possible.

de4dot — Déobfuscation automatique .NET

```
# De4dot — déobfuscation automatique

# Installation
# Télécharger depuis https://github.com/de4dot/de4dot

# Utilisation basique
de4dot malware_obfuscated.exe
# Génère malware_obfuscated-cleaned.exe avec les strings restaurées
# et les noms renommés en 'type_X' (pas les vrais noms, mais plus lisible)

# Forcer le détection d'un obfuscateur spécifique
de4dot --detect malware.exe          # Détecter l'obfuscateur
de4dot --cf malware.exe              # Forcer ConfuserEx

# Pour plusieurs fichiers
de4dot *.exe

# Résultat dans dnSpy après de4dot :
# - Strings déchiffrées et visibles
# - Classes renommées type_0001 au lieu de symboles unicode
# - Control flow un peu simplifié

# MAIS : de4dot ne déobfusque pas entièrement le control flow
# Pour ça : outils plus spécialisés comme ConfuserEx Deobfuscator
# ou analyse manuelle avec dnSpy debugger
```

Quand de4dot ne suffit pas — analyse manuelle du flow obfusqué

ConfuserEx 'mode fort' transforme le control flow en une machine à états. Le pattern typique : une variable locale (le 'switcher') est initialisée à une valeur, puis un switch géant choisit quel bloc de code exécuter selon cette valeur, chaque bloc modifiant le switcher pour passer au bloc suivant. Le vrai flow d'exécution est caché dans cette séquence de valeurs.

Stratégie : utilise le debugger dnSpy. Met un breakpoint sur la méthode obfusquée. Exécute instruction par instruction en observant quels blocs sont exécutés dans l'ordre. Note les valeurs importantes

(strings déchiffrées, appels API) au fur et à mesure. Tu n'as pas besoin de comprendre toute la machine à états — tu as besoin de comprendre CE QUE FAIT la méthode, pas comment le flow est géré.

3.7 Cas pratique : analyser AsyncRAT

AsyncRAT est un RAT .NET open source massivement utilisé par des groupes criminels. Il est en C#, souvent obfusqué avec ConfuserEx, et son code source est disponible sur GitHub (versions légitimes/éducatives). C'est un excellent sujet d'étude.

Les éléments à chercher dans AsyncRAT après déobfuscation :

AsyncRAT — Structure et IOCs à extraire

```
// Dans dnSpy, après de4dot, cherche ces classes/méthodes clés :

// 1. La classe Settings (ou Config) — contient la config C2
class Settings {
    public static string Host = Decrypt("EncryptedHost");   // IP/domaine C2
    public static string Port = Decrypt("EncryptedPort");   // Port
    public static string Version = "1.0.0";                 // Version du RAT
    public static string Mutex = Decrypt("EncryptedMutex"); // Mutex unique
    public static byte[] Key = ...;                          // Clé de chiffrement comm
s
}

// 2. La classe Client/Connection — communication avec le C2
// Cherche : new TcpClient(host, port) ou WebSocket
// AsyncRAT utilise AES-128-CBC pour chiffrer les communications
// La clé déchiffrée dans Settings.Key = IOC important

// 3. Les handlers de commandes — capabilities du RAT
// Switch statement ou Dictionary<string, Handler>
// Chaque case = une fonctionnalité :
// 'KeyLogger', 'FileManager', 'Processes', 'Shell', 'Webcam', etc.

// 4. Le mécanisme de persistence
// Généralement dans une classe Startup ou Persistence
// Registry Run Key ou Scheduled Task

// IOCs à extraire d'AsyncRAT :
// - IP/domaine C2 et port (dans Settings.Host / Settings.Port)
// - Nom du mutex (Settings.Mutex) — très stable entre variantes
// - Clé AES (Settings.Key) — unique par build
// - GUID de l'implant (Settings.InstallName)
```

```
// - Version (Settings.Version) — pour identifier la variante
```

CHAPITRE 4

Shellcode — Analyse et Émulation

Extraire, analyser et émuler du shellcode — Meterpreter, Beacon, shellcodes custom

4.1 Qu'est-ce qu'un shellcode et comment le trouver

Un shellcode est du code machine autonome, conçu pour être injecté et exécuté dans n'importe quelle région de mémoire exécutable, sans dépendre d'un PE header. Contrairement à un EXE, un shellcode ne peut pas utiliser d'imports PE (il n'a pas de table d'imports). Il doit donc trouver lui-même les fonctions dont il a besoin, généralement via le PEB traversal que l'on a vu au volume 1.

Le shellcode est omniprésent dans l'écosystème malware. Les stagers Meterpreter sont des shellcodes. Les CobaltStrike Beacons sont souvent délivrés via un shellcode stager. Les exploits de navigateurs injectent des shellcodes dans le processus navigateur. Beaucoup de malwares déchiffrent et exécutent un shellcode en mémoire (d'où les appels VirtualAlloc + WriteProcessMemory que l'on a vus).

Où trouver le shellcode dans un binaire

Le shellcode ne se trouve pas nécessairement dans une section PE spécifique. Il peut être :

- Chiffré dans .rdata ou .data — déchiffré et copié en mémoire exécutable avant exécution
- Embarqué dans les ressources (.rsrc) — extrait et exécuté
- Téléchargé depuis un C2 et placé en mémoire via VirtualAlloc + WriteFile + CreateThread
- Dans une DLL injectée via reflective loading — pas de fichier sur disque
- Dans des streams OLE d'un document Office (.docm, .xls) avec macros

Dans Ghidra, le shellcode apparaît comme une zone de bytes dans la section de données qui ne ressemble pas à du texte ou des structures. Si l'entropie est élevée et si tu vois une fonction qui fait VirtualAlloc puis copie ces bytes vers la zone allouée → c'est un shellcode.

4.2 Extraire un shellcode d'un binaire

Une fois que tu as trouvé où est le shellcode (chiffré ou non), il faut l'extraire pour l'analyser séparément.

Méthode 1 : Extraction statique avec Python

Python — Extraction de shellcode depuis un PE

```
# Extraire et déchiffrer un shellcode depuis un PE en Python

import pefile
```

```
pe = pefile.PE('malware.exe')

# Trouver la section qui contient le shellcode
# (tu as trouvé l'offset dans Ghidra - par exemple .rdata à RVA 0x3000)
for section in pe.sections:
    if b'.rdata' in section.Name:
        raw_data = section.get_data()
        print(f'Section .rdata : {len(raw_data)} bytes')

# Extraire les bytes du shellcode
# (tu sais qu'il commence à l'offset 0x100 dans la section et fait 0x500 bytes)
shellcode_offset = 0x100
shellcode_size   = 0x500
encrypted_sc = raw_data[shellcode_offset:shellcode_offset + shellcode_size]

# Déchiffrer (clé XOR trouvée dans Ghidra = 0x41)
key = 0x41
shellcode = bytes(b ^ key for b in encrypted_sc)

# Vérifier : un shellcode x64 commence souvent par ces bytes caractéristiques
# Windows/x64 PEB traversal commence souvent par :
# 0x65 0x48 0x8B 0x04 0x25 0x60 0x00 0x00 0x00 = MOV RAX, GS:[0x60]
if shellcode[:3] == b'\xfc\x48\x83':
    print('Probable shellcode Meterpreter x64 !')
elif shellcode[:2] == b'\x4d\x5a':
    print('Shellcode contient un PE embarqué (Reflective DLL ?)')

# Sauvegarder pour analyse
with open('shellcode_extracted.bin', 'wb') as f:
    f.write(shellcode)
print(f'Shellcode sauvegardé : {len(shellcode)} bytes')
```

Méthode 2 : Extraction dynamique avec x64dbg

La méthode dynamique est souvent plus fiable : tu laisses le malware déchiffrer lui-même le shellcode en mémoire, tu poses un breakpoint juste avant l'appel à CreateThread/CreateRemoteThread, et tu dumpes la région mémoire depuis Process Hacker ou directement depuis x64dbg.

Extraction dynamique de shellcode

```
; Dans x64dbg : extraire le shellcode après déchiffrement en mémoire

; 1. Pose un BP sur CreateThread
bp CreateThread
```

```
; 2. F9 → le programme déchiffre le shellcode, puis s'arrête au BP
; 3. À ce point, le shellcode est en mémoire déchiffré
;    Regarde le 3ème argument (lpStartAddress = R8 en Windows x64)
;    C'est l'adresse de début du shellcode
; 4. Dans x64dbg Memory Map (Alt+M), trouve la région qui contient cette adresse
;    Elle devrait avoir le flag PAGE_EXECUTE_READWRITE ou PAGE_EXECUTE_READ
; 5. Clic droit sur cette région → Dump Memory to File
;    Sauvegarde le fichier .bin
; 6. Ouvre ce .bin dans Ghidra ou avec scDbg pour l'analyser
; Alternative : depuis Process Hacker
; Onglet Memory → trouver la région rwx privée
; Clic droit → Save to File
```

4.3 Analyser un shellcode dans Ghidra

Ghidra peut analyser des raw binaries (pas des PE). Lors de l'import : Format → Raw Binary, Language → x86:LE:64:default. Ghidra va désassembler depuis le byte 0. Attention : il ne connaît pas l'ImageBase réelle du shellcode — les adresses seront relatives à 0x0 (ou à l'adresse que tu lui donnes à l'import).

Le début d'un shellcode x64 typique ressemble à du code bizarre parce qu'il commence souvent par trouver le kernel32.dll via le PEB traversal. Reconnaître ce pattern te dit immédiatement que c'est du shellcode Windows. Après la phase de résolution de fonctions, le shellcode appelle les WinAPIs normalement.

Shellcode x64 — Anatomie du début

```
; Début typique d'un shellcode x64 Windows (pattern très courant)
; Phase 1 : Trouver l'adresse de base de kernel32.dll via le PEB
cld                             ; Clear Direction Flag (nettoyage)
mov  rbp, rsp
and  rsp, 0xFFFFFFFFFFFFFFF0    ; Aligner la stack sur 16 bytes
sub  rsp, 0x20                  ; Shadow space
xor  rdx, rdx
mov  rdx, gs:[rdx+0x60]         ; RDX = PEB
mov  rdx, [rdx+0x18]            ; RDX = PEB.Ldr
mov  rdx, [rdx+0x20]            ; RDX = InMemoryOrderModuleList.Flink
mov  rdx, [rdx]                 ; Skip first entry (exe lui-même)
mov  rdx, [rdx]                 ; Skip second entry (ntdll.dll)
```

```
mov  rsi, [rdx+0x20]            ; RSI = DllBase de kernel32.dll

; Phase 2 : Parser l'Export Directory de kernel32.dll pour GetProcAddress
; (longue séquence de parsing PE pour trouver GetProcAddress)
; Puis appelle GetProcAddress pour charger les autres APIs

; Phase 3 : Payload réel (ex: reverse shell TCP)
; Appelle les WinAPIs trouvées dynamiquement :
; WSASocket → connect → CreateProcess(cmd.exe) → redirection IO

; Dans Ghidra, le décompilateur génère quelque chose comme :
; undefined8 * lVar1 = *(undefined8 *)(*(long *)(__readgsqword(0x60) + 0x18) + 0x20);
; → C'est le PEB traversal ! Reconnaissable même obfusqué.
```

4.4 Émuler un shellcode avec Unicorn Engine

Unicorn Engine est un framework d'émulation CPU. Il te permet d'exécuter du code machine dans un environnement contrôlé sans vraiment exécuter le code sur ton CPU. L'avantage : tu contrôles tout — mémoire, registres, interruptions — et tu peux observer chaque instruction sans avoir besoin d'un debugger.

Pour l'analyse de shellcode, Unicorn te permet de : déchiffrer/dépacker un shellcode stub qui déchiffre lui-même son payload, extraire les strings qui sont construites dynamiquement en mémoire, et bypasser les anti-analyses qui détectent x64dbg (Unicorn n'est pas un debugger Windows, il est invisible aux checks classiques).

Unicorn Engine — Émulation de shellcode x64

```
# Unicorn Engine — Émuler un shellcode x64
# pip install unicorn --break-system-packages

from unicorn import *
from unicorn.x86_const import *

# Charger le shellcode extrait
with open('shellcode_extracted.bin', 'rb') as f:
    shellcode = f.read()

# Créer l'émulateur x64
mu = Uc(UC_ARCH_X86, UC_MODE_64)

# Mapper la mémoire
BASE_ADDR = 0x10000000  # Adresse arbitraire pour le shellcode
STACK_ADDR = 0x00100000  # Stack
STACK_SIZE = 0x100000    # 1 Mo de stack
MEM_SIZE   = 0x100000    # 1 Mo pour le code
```

```
mu.mem_map(BASE_ADDR, MEM_SIZE)
mu.mem_map(STACK_ADDR, STACK_SIZE)

# Écrire le shellcode en mémoire
mu.mem_write(BASE_ADDR, shellcode)

# Initialiser les registres
mu.reg_write(UC_X86_REG_RSP, STACK_ADDR + STACK_SIZE // 2)  # Stack au milieu
mu.reg_write(UC_X86_REG_RBP, STACK_ADDR + STACK_SIZE // 2)

# Hook sur chaque instruction pour observer l'exécution
executed_instrs = []

def hook_code(uc, address, size, user_data):
    # Logger l'adresse et les bytes de chaque instruction
    bytes_at_addr = bytes(uc.mem_read(address, min(size, 15)))
    rax = uc.reg_read(UC_X86_REG_RAX)
    rcx = uc.reg_read(UC_X86_REG_RCX)
    executed_instrs.append((address, bytes_at_addr, rax, rcx))

def hook_mem_read(uc, access, address, size, value, user_data):
    # Logger les accès mémoire intéressants
    if address == 0x60:  # Accès au PEB pointer !
        print(f'PEB access détecté @ {hex(uc.reg_read(UC_X86_REG_RIP))}')
        # Simuler le PEB en retournant une valeur factice
        # (pour que l'émulation continue sans crash)
        uc.reg_write(UC_X86_REG_RAX, 0x00DEAD00)  # Fake PEB address

mu.hook_add(UC_HOOK_CODE, hook_code)
mu.hook_add(UC_HOOK_MEM_READ_UNMAPPED, hook_mem_read)

# Lancer l'émulation (max 1000 instructions pour commencer)
try:
    mu.emu_start(BASE_ADDR, BASE_ADDR + len(shellcode), timeout=5000, count=1000)
except UcError as e:
    print(f'Émulation arrêtée : {e}')
    print(f'Dernière adresse : {hex(mu.reg_read(UC_X86_REG_RIP))}')

print(f'Instructions émulées : {len(executed_instrs)}')

# Chercher des strings créées en mémoire pendant l'émulation
# (souvent les shellcodes construisent les strings API dynamiquement)
mem_content = bytes(mu.mem_read(STACK_ADDR, STACK_SIZE))
# Chercher des strings ASCII dans la mémoire post-émulation
```

```
import re
strings = re.findall(b'[\x20-\x7e]{5,}', mem_content)
for s in strings:
    print(f'String trouvée en mémoire : {s.decode()}')
```

4.5 Identifier et extraire la config d'un Beacon CobaltStrike

CobaltStrike Beacon est l'un des RATs/C2 frameworks les plus utilisés par les groupes APT et les criminels. Sa configuration (C2 address, User-Agent, sleep time, etc.) est embarquée dans le Beacon et chiffrée avec XOR 0x69 (version non-patché) ou avec une clé customisée selon la version.

La bonne nouvelle : la structure de configuration du Beacon est bien documentée par la communauté RE (notamment par Didier Stevens et le projet BeaconHunter). Il y a des outils qui l'extractent automatiquement.

Extraction config CobaltStrike Beacon

```
# Extraction config CobaltStrike Beacon — méthode simplifiée
# Le Beacon CS a une signature reconnue et une structure de config documentée

import struct, re

# Structure d'une entrée de config CS Beacon
# Les settings sont des TLVs : Type(2B) + Length(2B) + Value(Length B)

BEACON_CONFIG_SETTINGS = {
    0x0001: 'BeaconType',          # 0=HTTP, 1=Hybrid, 8=HTTPS
    0x0002: 'Port',
    0x0003: 'SleepTime',           # Intervalle beacon en ms
    0x0004: 'MaxGetSize',
    0x0005: 'Jitter',              # Jitter en %
    0x0007: 'C2Server',            # Host,URL
    0x000B: 'UserAgent',           # HTTP User-Agent (IOC !)
    0x000C: 'HttpPostURI',         # URI pour les POSTs
    0x001A: 'Watermark',           # ID opérateur (IOC pour attribution)
    0x001D: 'ProcInject_Stub',
    0x0035: 'HttpGet_Verb',
    0x0036: 'HttpPost_Verb',
}

def parse_beacon_config(data):
    """Parser le bloc de config Beacon déchiffré"""
    i = 0
```

```
    results = {}
    while i < len(data) - 4:
        setting_type = struct.unpack_from('>H', data, i)[0]
        length = struct.unpack_from('>H', data, i+2)[0]
        if setting_type == 0 or length == 0:
            i += 4
            continue
        value_bytes = data[i+4:i+4+length]
        name = BEACON_CONFIG_SETTINGS.get(setting_type, f'Setting_0x{setting_type:04X}'
)
        # Détecter le type de valeur
        if length == 2:
            value = struct.unpack('>H', value_bytes)[0]
        elif length == 4:
            value = struct.unpack('>I', value_bytes)[0]
        else:
            value = value_bytes.rstrip(b'\x00').decode('utf-8', errors='replace')
        results[name] = value
        i += 4 + length
    return results

def extract_cs_config(beacon_path):
    with open(beacon_path, 'rb') as f:
        data = f.read()

    # Chercher le magic bytes du bloc de config
    # CS Beacon config commence par un padding de 0x00 puis les settings
    # Essayer XOR 0x69 (version classique)
    for offset in range(len(data) - 4):
        # Chercher le pattern 0x0001 0x0002 (premiers settings)
        candidate = bytes(b ^ 0x69 for b in data[offset:offset+4])
        if candidate == b'\x00\x01\x00\x02':
            # Probable début de config !
            config_data = bytes(b ^ 0x69 for b in data[offset:offset+4096])
            config = parse_beacon_config(config_data)
            return config
    return None

config = extract_cs_config('beacon_sample.bin')
if config:
```

```
    for k, v in config.items():
        print(f'{k}: {v}')
# Exemple de sortie :
# C2Server: 185.220.101.47,/jquery-3.3.1.min.js
# UserAgent: Mozilla/5.0 (Windows NT 6.3; Trident/7.0; rv:11.0) like Gecko
# SleepTime: 60000
# Jitter: 10
# Watermark: 1873433027   ← ID opérateur CobaltStrike !
```

CHAPITRE 5

Identifier les Algorithmes Cryptographiques

Reconnaître RC4, AES, RSA et d'autres en assembleur — les décrypter offline

5.1 Pourquoi identifier la crypto est crucial

Les malwares utilisent la cryptographie de deux façons principales : pour chiffrer les communications avec le C2 (tu ne peux pas lire le trafic sans la clé), et pour chiffrer la config et les strings internes (tu ne peux pas extraire les IOCs sans déchiffrer). Identifier correctement l'algorithme utilisé te permet d'écrire le déchiffreur qui te donnera ces informations.

Beaucoup de malwares n'utilisent pas les APIs Windows pour la crypto (CryptEncrypt, BCryptEncrypt) parce que ça crée des imports visibles. À la place, ils implémentent les algorithmes en pur C/assembleur. C'est pour ça que tu dois reconnaître les patterns de code des algorithmes courants.

5.2 Reconnaître RC4

RC4 est l'algorithme de chiffrement le plus implémenté dans les malwares parce qu'il est simple (une vingtaine de lignes de code), rapide, et suffisant. Il a deux phases : le Key Scheduling Algorithm (KSA) et le Pseudo-Random Generation Algorithm (PRGA).

Le signal RC4 dans Ghidra : cherche une initialisation d'un tableau de 256 éléments suivie d'une double boucle. La première boucle ({ic('for i in range(256)')} avec {ic('S[i] = i')}) est le KSA init. La deuxième boucle avec un swap d'éléments et XOR est le KSA + PRGA. Le pattern est très reconnaissable même en assembleur.

RC4 KSA — signature en assembleur

```
; RC4 KSA (Key Scheduling Algorithm) en assembleur — très reconnaissable

; Phase 1 : Initialisation du tableau S[256]
; for (i=0; i<256; i++) S[i] = i;
lea  rbx, [S_array]       ; RBX = pointeur vers S[256]
xor  rcx, rcx             ; i = 0
ksa_init:
mov  byte [rbx+rcx], cl   ; S[i] = i (octet bas de i)
inc  rcx
cmp  rcx, 256
jl   ksa_init

; Phase 2 : Mélange avec la clé
; j=0; for (i=0; i<256; i++) { j=(j+S[i]+key[i%keylen])%256; swap(S[i],S[j]); }
```

```
xor  rdx, rdx            ; j = 0
xor  rsi, rsi            ; i = 0
ksa_mix:
movzx eax, byte [rbx+rsi]       ; eax = S[i]
add   rdx, rax                  ; j += S[i]
; ... calcul de key[i % keylen] ...
add   rdx, rax                  ; j += key[i%keylen]
and   rdx, 0xFF                 ; j %= 256
; SWAP S[i] et S[j]
movzx eax, byte [rbx+rsi]       ; tmp = S[i]
movzx ecx, byte [rbx+rdx]       ; S[i] = S[j]
mov   byte [rbx+rsi], cl
mov   byte [rbx+rdx], al        ; S[j] = tmp
inc   rsi
cmp   rsi, 256
jl    ksa_mix

; → Ce double tableau à 256 éléments + les deux boucles = signature RC4
; → La clé RC4 est chargée dans la première phase → IOC important !
```

5.3 Reconnaître AES

AES est plus complexe que RC4 et son implémentation en assembleur est plus longue. Les signatures AES les plus fiables sont les constantes mathématiques de l'algorithme : la S-Box (un tableau de 256 bytes avec des valeurs spécifiques), et les round constants (Rcon). Ces constantes sont identiques dans toutes les implémentations AES correctes.

Dans Ghidra : cherche un tableau de 256 bytes avec ces valeurs de début : {ic('0x63 0x7c 0x77 0x7b 0xf2 0x6b 0x6f 0xc5 0x30 0x01 0x67 0x2b 0xfe 0xd7...')} C'est la S-Box AES. Si tu trouves ce tableau dans .rdata ou .data → le programme utilise AES.

Détecter AES dans un binaire

```
# Détecter AES dans un binaire avec Python

# La S-Box AES — tous les bytes dans cet ordre
AES_SBOX = bytes([
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5,
    0x30, 0x01, 0x67, 0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0,
    # ... 256 bytes au total
])
```

```
with open('malware.exe', 'rb') as f:
    data = f.read()

# Chercher la S-Box dans le binaire
idx = data.find(AES_SBOX[:16])  # Chercher les 16 premiers bytes
if idx != -1:
    print(f'S-Box AES trouvée à l offset 0x{idx:X}')
    print('→ Le malware utilise AES implémenté en pur code')

# Chercher aussi les appels BCryptEncrypt (AES via API Windows)
import pefile
pe = pefile.PE('malware.exe')
for entry in pe.DIRECTORY_ENTRY_IMPORT:
    for imp in entry.imports:
        if imp.name and b'Crypt' in imp.name:
            print(f'Import crypto trouvé : {imp.name.decode()}')
```

5.4 FindCrypt — automatiser dans Ghidra

FindCrypt est un plugin Ghidra (et IDA) qui scan automatiquement le binaire à la recherche de constantes cryptographiques : S-Box AES, constantes MD5/SHA, clés DES prédéfinies, etc. Il annote directement les emplacements trouvés dans Ghidra.

Installation : télécharge FindCrypt depuis GitHub (ghidra_FindCrypt ou FindCrypt-Ghidra), installe comme extension Ghidra. Lance depuis Script Manager → FindCrypt.py. En quelques secondes tu vois : '{ic('AES SBox at 0x140004120')}, {ic('MD5 Init at 0x14000A230')}'. Navigue directement vers ces adresses pour trouver les fonctions crypto.

5.5 Décrypter une config malware étape par étape

Voici un walkthrough complet d'extraction et déchiffrement d'une config malware réelle. Le scénario : tu as un sample inconnu, tu sais qu'il a une config C2 quelque part.

Walkthrough complet — extraction et déchiffrement config

```
; Étape 1 : Dans Ghidra, cherche la fonction d'initialisation
; Pattern à chercher : fonction appelée tôt dans main(), qui charge des données
; et appelle une fonction de déchiffrement

; Dans le pseudocode Ghidra :
; void init_config(void) {
;     local_c2 = &DAT_140008000;    // Pointeur vers données chiffrées
;     local_len = 0x64;             // Longueur = 100 bytes
```

```
;     local_key = 0x2B;             // Clé XOR = 0x2B
;     decrypt_xor(local_c2, local_len, local_key);  // Déchiffrement
;     // Maintenant DAT_140008000 contient la config déchiffrée
; }

; Étape 2 : Extraire les données chiffrées depuis Ghidra
; Navigue vers DAT_140008000 dans le Listing
; Sélectionne 0x64 bytes
; Clic droit → Copy Special → Byte String → Colle dans Python

; Étape 3 : Déchiffrer en Python
encrypted = bytes([0x59, 0x4E, 0x58, 0x59, 0x5F, 0x18, 0x56, ...])
key = 0x2B
decrypted = bytes(b ^ key for b in encrypted)

; Étape 4 : Parser la config déchiffrée
; Regarde comment la config est utilisée dans le code :
; - DAT_140008000 est passé à InternetConnectW → c'est le host C2
; - DAT_140008040 (offset 0x40 dans la config) est passé à Sleep → sleep timer

c2_host = decrypted[:64].rstrip(b'\x00').decode('utf-16-le')
sleep_ms = int.from_bytes(decrypted[64:68], 'little')

print(f'C2 Host : {c2_host}')
print(f'Sleep   : {sleep_ms} ms ({sleep_ms//1000}s)')
```

CHAPITRE 6

VM Protectors et Obfuscation Avancée

Quand VMProtect et OLLVM rendent le décompilateur inutile — stratégies alternatives

6.1 Code virtualization : comment VMProtect fonctionne

VMProtect et Themida sont des protecteurs commerciaux qui appliquent une technique appelée code virtualization. Au lieu de laisser le code machine x86/x64 original (que Ghidra peut décompiler), ils le remplacent par un bytecode custom pour une VM qui n'existe que dans ce programme. Le code original est transformé en une séquence d'opcodes pour la VM interne. La VM (écrite en x86/x64) interprète ces opcodes.

Ce que ça signifie pour toi : Ghidra voit le code de la VM (l'interpréteur), pas le code original. Le décompilateur génère du pseudocode qui décrit l'interpréteur, pas la logique métier originale. Tu peux avoir une fonction de 1000 lignes de pseudocode qui correspond à {ic('if (x > 5) return 1;')} dans le code original.

Comment reconnaître un binaire virtualisé

DIE signale : `VMProtect 3.x.x` ou `Themida 3.x.x`. Dans Ghidra : une fonction gigantesque au point d'entrée avec des centaines de cases dans un switch géant. Des chaînes de dispatch incompréhensibles. L'entropie est haute mais pas dans les sections PE normales (dans le code lui-même). Aucune des fonctions ne ressemble à du code 'normal'.

6.2 Stratégies d'analyse pour les protecteurs forts

Face à VMProtect ou Themida, l'analyse statique avec Ghidra est quasi-inutile. Tu dois changer d'approche.

Stratégie 1 — Analyse dynamique pure. Même avec la virtualisation, le malware doit appeler des WinAPIs au final. La virtualisation ne s'applique généralement qu'à une partie du code (les parties critiques de protection et de logique core). Les fonctions moins importantes (parsing réseau, persistence) restent souvent en code natif. Utilise x64dbg avec BPs sur les APIs pour observer le comportement sans déchiffrer la VM.

Stratégie 2 — Analyse du trafic réseau. Un malware protégé doit quand même communiquer. FakeNet-NG + Wireshark te donnent les IOCs réseau (IP, domaine, URL, User-Agent) même si tu ne comprends rien au code.

Stratégie 3 — Sandbox automatique. CAPE Sandbox ou ANY.RUN font une analyse comportementale sans essayer de déchiffrer la VM. Ils te donnent les artefacts système, les connexions réseau, les fichiers créés.

Stratégie 4 — Déprotection (avancé). VMProtect a des vulnérabilités connues qui permettent de partiellement déprotéger certaines versions. Des outils comme VMPDump peuvent dumper le code déprotégé depuis la mémoire. C'est une spécialité à part entière — des chercheurs passent des semaines sur un seul binaire VMProtect.

6.3 OLLVM — Obfuscation au niveau compilateur

OLLVM (Obfuscator-LLVM) est un fork de LLVM qui applique des transformations d'obfuscation pendant la compilation. Contrairement aux protecteurs post-compilation, OLLVM transforme l'IR LLVM (représentation intermédiaire) avant de générer le code machine. Le résultat est du code machine 'légitimement' généré mais incompréhensible.

Les 3 transformations principales d'OLLVM :

- **Control Flow Flattening** : le flow d'une fonction est aplati en une machine à états géante. Tout passe par un dispatcher central. Impossible à lire directement dans Ghidra.
- **Bogus Control Flow** : des branches mortes (code qui ne s'exécute jamais) sont ajoutées avec des conditions opaques (toujours vraies ou fausses mais indécidables statiquement).
- **Instruction Substitution** : les opérations simples (ADD, XOR) sont remplacées par des séquences équivalentes mais plus complexes. `a + b` devient `a - (-b)` ou pire.

Déobfusquer OLLVM avec Miasm

Miasm est un framework RE Python qui permet notamment de faire de l'exécution symbolique et de simplifier des blocs de code. Il peut partiellement résoudre le Control Flow Flattening d'OLLVM en identifiant les conditions opaques et les supprimant.

Miasm — Analyse de code OLLVM

```
# Miasm — Simplifier le control flow flattening OLLVM
# pip install miasm --break-system-packages

from miasm.analysis.machine import Machine
from miasm.core.locationdb import LocationDB
from miasm.analysis.binary import Container
from miasm.analysis.depgraph import DependencyGraph

# Charger le binaire
with open('ollvm_malware.exe', 'rb') as f:
    cont = Container.from_stream(f)

machine = Machine('x86_64')
mdis = machine.dis_engine(cont.bin_stream, loc_db=LocationDB())

# Désassembler depuis l'adresse de la fonction OLLVM
func_addr = 0x140003000  # Adresse trouvée dans Ghidra
```

```
asmcfg = mdis.dis_multiblock(func_addr)

# Analyser le graphe de contrôle
print(f'Blocs de base : {len(asmcfg.blocks)}')
# Un nombre très élevé de blocs = probable OLLVM flattening

# Pour la simplification complète :
# Voir les outils spécialisés :
# - D810 (plugin IDA) : https://github.com/joydo/d810
# - deflat.py : script Python pour OLLVM CFF
# - SATURN : déobfuscateur symbolique

# Méthode pragmatique pour le SOC :
# Si la fonction est protégée par OLLVM mais appelle des APIs :
# → BP dynamique sur les APIs en x64dbg
# → Logger les arguments à chaque appel
# → Tu comprends le comportement sans déchiffrer le flow
```

CHAPITRE 7

Malwares Avancés et Techniques Modernes

Fileless, reflective loading, BOF, rootkits — les techniques d'élite

7.1 Fileless malware : code en mémoire uniquement

Un malware fileless ne dépose pas de fichier .exe sur le disque. Il vit entièrement en mémoire. Ça ne veut pas dire qu'il est invisible : il utilise toujours des processus Windows légitimes, crée des clés de registre, et laisse des traces dans les logs Sysmon. Mais les AV basés sur l'analyse de fichiers ne peuvent pas le détecter.

Le vecteur typique d'un malware fileless : une macro Office télécharge un script PowerShell en base64 encodé, PowerShell décode et exécute le script en mémoire ({ic('-EncodedCommand')}), le script PowerShell télécharge un shellcode depuis le C2, injecte le shellcode dans un processus légitime via {ic('VirtualAlloc + CreateThread')}, et le shellcode (Beacon, Meterpreter...) s'exécute dans ce processus.

Analyse d'un malware fileless : tu n'as pas de fichier EXE à analyser dans Ghidra. Tu analyses le script (PowerShell, VBScript, JScript), tu extrais le shellcode embarqué (souvent en base64 dans le script), et tu analyses le shellcode avec Ghidra ou scDbg.

Analyser un script PowerShell malveillant

```
# Déobfusquer et analyser un script PowerShell malveillant

# Le script typiquement reçu par email :
# powershell.exe -nop -w hidden -EncodedCommand SQBFAFgAIAAoA...

import base64, re

# Décoder la commande base64
encoded = 'SQBFAFgAIAAoAE4AZQB3AC0ATwBiAGoAZQBjAHQAIABOAGUAdAAuAFcA...'
decoded = base64.b64decode(encoded).decode('utf-16-le', errors='replace')
print('Script PowerShell décodé :')
print(decoded[:500])

# Chercher des patterns dans le script
# URL de téléchargement du payload
urls = re.findall(r'https?://[^\s\'"]+', decoded)
print(f'URLs trouvées : {urls}')

# Payloads base64 embarqués dans le script
b64_patterns = re.findall(r"'([A-Za-z0-9+/]{100,}={0,2})'", decoded)
for b64 in b64_patterns:
```

```
    try:
        payload = base64.b64decode(b64)
        if payload[:2] == b'MZ':
            print(f'PE embarqué trouvé ! Taille : {len(payload)} bytes')
            with open('embedded_pe.exe', 'wb') as f:
                f.write(payload)
        elif payload[:4] == b'\xfc\xe8\x82':  # Magic bytes shellcode commun
            print(f'Shellcode trouvé ! Taille : {len(payload)} bytes')
            with open('shellcode.bin', 'wb') as f:
                f.write(payload)
    except:
        pass

# IOCs à extraire d'un PowerShell malveillant :
# - URLs de téléchargement (C2 de staging)
# - Payloads embarqués (DLL, EXE, shellcode)
# - Variables d'environnement accédées (recon)
# - Cibles d'injection (processus cible)
# - Techniques d'évasion (AMSI bypass, WD exclusion)
```

7.2 Reflective DLL Loading — anatomie complète

La reflective DLL loading est une technique qui permet à une DLL de se charger elle-même en mémoire sans passer par le loader Windows normal. Résultat : la DLL n'apparaît pas dans la liste des modules chargés (PEB), il n'y a aucun fichier DLL sur le disque, et les outils qui listent les DLLs chargées ne la voient pas.

Le mécanisme : une DLL réflective contient une fonction exportée spéciale (souvent {ic('ReflectiveDllInjection')} ou un nom obfusqué) qui, quand elle est appelée avec l'adresse de la DLL en mémoire comme argument, effectue elle-même toutes les opérations que le loader Windows fait normalement : parser le PE header, mapper les sections, résoudre les imports, appliquer les relocations, et appeler le DllMain. C'est une implémentation du loader PE dans la DLL elle-même.

Détecter une reflective DLL en cours d'exécution

Dans Process Hacker : l'onglet Memory d'un processus suspect montre des régions de type {ic('Private')} (pas mappées depuis un fichier) qui ont le flag executable. Si le Path est vide et la région est private + executable → probable code injecté. Dump cette région et analyse-la dans Ghidra.

Détection de régions mémoire suspectes

```
# Détecter les DLLs réflectives avec Python + psutil
import psutil, ctypes, os
```

```
from ctypes.wintypes import HANDLE

# Pour chaque processus en cours d'exécution
for proc in psutil.process_iter(['pid', 'name', 'exe']):
    try:
        # Ouvrir le processus
        pid = proc.info['pid']

        # Les vraies DLLs ont un chemin sur disque
        # Les DLLs réflectives : chemin vide ou incohérent
        for module in proc.memory_maps():
            if not module.path and 'x' in module.perms:
                # Région exécutable sans fichier source → suspect
                print(f'[PID {pid}] Région privée exécutable : {module.addr}')
                print(f'  Processus : {proc.info["name"]}')
                print(f'  Taille : {module.rss} bytes')
                print(f'  → Possible code injecté ou reflective DLL')
    except (psutil.NoSuchProcess, psutil.AccessDenied):
        pass
```

7.3 Living off the Land Binaries (LOLBins)

Les LOLBins sont des binaires Windows légitimes (signés Microsoft) qui peuvent être utilisés à des fins malveillantes. En les utilisant, l'attaquant évite de déposer un EXE malveillant sur le disque — le programme qui s'exécute est légitime, c'est son usage qui est malveillant.

Connaître les LOLBins est crucial pour le RE car tu verras souvent un malware qui lance ces binaires plutôt qu'un EXE custom. Les LOLBins les plus courants en malwares :

- `mshta.exe` : Exécute des fichiers HTA (HTML Application). Souvent utilisé pour exécuter du VBScript/JavaScript malveillant.
- `regsvr32.exe` : Enregistre des DLLs COM. Peut charger une DLL depuis une URL HTTP (scrobj.dll + /u flag).
- `certutil.exe` : Peut décoder du base64 et télécharger des fichiers. Très utilisé pour downloader des payloads.
- `rundll32.exe` : Exécute des exports de DLL. Utilisé pour lancer des DLLs malveillantes sans processus séparé.
- `powershell.exe` : Exécute des scripts, peut downloader et exécuter en mémoire. La LOLBin reine.
- `wmic.exe` : Interface WMI. Peut exécuter des commandes localement et à distance.
- `msiexec.exe` : Installateur MSI. Peut installer depuis des URLs et exécuter des code custom.
- `bitsadmin.exe` : Gestionnaire BITS. Télécharge des fichiers en arrière-plan depuis des URLs.

Pour les règles SIEM : surveiller ces processus avec des arguments inhabituels. Ex : `certutil.exe -urlcache -split -f http://...` = téléchargement de payload. `regsvr32.exe /s /n /u /i:http://...` = exécution scriptlet depuis URL.

CHAPITRE 8

Automatiser l'Analyse RE

De CAPE Sandbox aux scripts Python — construire un pipeline d'analyse

8.1 CAPE Sandbox — la sandbox de référence

CAPE (Config And Payload Extraction) est une sandbox de malware analyse basée sur Cuckoo Sandbox. Sa spécialité : en plus du rapport comportemental standard (processus créés, fichiers, registre, réseau), elle extrait automatiquement les configs de malwares connus (RedLine, AgentTesla, Formbook, CobaltStrike Beacon, etc.) en utilisant des extracteurs spécifiques par famille.

Tu peux utiliser la version publique (capesandbox.com) pour des analyses rapides, ou déployer ta propre instance dans ton lab. L'API REST te permet d'automatiser les soumissions et la récupération des résultats.

CAPE Sandbox API — Pipeline automatisé

```
# CAPE Sandbox API — automatiser la soumission et récupération
import requests, time, json

CAPE_URL = 'http://your-cape-instance:8000'
API_KEY  = 'your_api_key'

def submit_sample(filepath):
    """Soumet un sample à CAPE et retourne le task ID"""
    with open(filepath, 'rb') as f:
        response = requests.post(
            f'{CAPE_URL}/apiv2/tasks/create/file/',
            headers={'Authorization': f'Token {API_KEY}'},
            files={'file': f},
            data={'options': 'procmemdump=1,extraction=1'}  # Dump mémoire + extraction
        )
    return response.json()['data']['task_ids'][0]

def wait_and_get_report(task_id, timeout=300):
    """Attend la fin de l'analyse et récupère le rapport"""
    for _ in range(timeout // 10):
        time.sleep(10)
        status = requests.get(
            f'{CAPE_URL}/apiv2/tasks/status/{task_id}/',
            headers={'Authorization': f'Token {API_KEY}'}
```

```
        ).json()['data']['status']
        if status == 'reported':
            report = requests.get(
                f'{CAPE_URL}/apiv2/tasks/get/report/{task_id}/',
                headers={'Authorization': f'Token {API_KEY}'}
            ).json()
            return report
    return None

def extract_iocs_from_cape(report):
    """Extraire les IOCs du rapport CAPE"""
    iocs = {
        'hashes': [],
        'network': {'ips': [], 'domains': [], 'urls': []},
        'registry': [],
        'files': [],
        'config': {}
    }

    # Hashes
    if 'target' in report:
        t = report['target'].get('file', {})
        iocs['hashes'] = {
            'md5':    t.get('md5'),
            'sha256': t.get('sha256'),
        }

    # Réseau
    network = report.get('network', {})
    iocs['network']['ips']     = list({h['dst'] for h in network.get('hosts', [])})
    iocs['network']['domains'] = [d['domain'] for d in network.get('domains', [])]
    iocs['network']['urls']    = [r['uri'] for r in network.get('http', [])]

    # Config extraite par CAPE (si famille connue)
    for proc in report.get('CAPE', {}).get('configs', []):
        iocs['config'].update(proc)

    return iocs

# Utilisation
task_id = submit_sample('/tmp/suspicious.exe')
print(f'Task soumis : {task_id}')
```

```
report = wait_and_get_report(task_id)
if report:
    iocs = extract_iocs_from_cape(report)
    print(json.dumps(iocs, indent=2))
```

8.2 Écrire un extracteur d'IOCs générique en Python

Un extracteur d'IOCs Python automatise le triage statique : il calcule les hashes, parse le PE, extrait les strings, et produit un rapport structuré en quelques secondes. Voici un extracteur complet à utiliser comme base.

ioc_extractor.py — Extracteur automatique complet

```
#!/usr/bin/env python3
# ioc_extractor.py — Extracteur d'IOCs statique pour malware EXE
# pip install pefile yara-python python-magic --break-system-packages

import hashlib, pefile, subprocess, json, re, sys
from pathlib import Path

def compute_hashes(filepath):
    data = Path(filepath).read_bytes()
    return {
        'md5':    hashlib.md5(data).hexdigest(),
        'sha1':   hashlib.sha1(data).hexdigest(),
        'sha256': hashlib.sha256(data).hexdigest(),
    }

def pe_analysis(filepath):
    try:
        pe = pefile.PE(filepath)
    except Exception as e:
        return {'error': str(e)}

    result = {
        'arch': 'x64' if pe.FILE_HEADER.Machine == 0x8664 else 'x86',
        'entrypoint': hex(pe.OPTIONAL_HEADER.AddressOfEntryPoint),
        'imagebase':  hex(pe.OPTIONAL_HEADER.ImageBase),
        'compile_ts': pe.FILE_HEADER.TimeDateStamp,
        'imphash':    pe.get_imphash(),
        'sections':   [],
        'imports':    {},
```

```
        'suspicious_imports': [],
    }

    # Sections avec entropie
    for s in pe.sections:
        name = s.Name.decode(errors='replace').strip('\x00')
        ent  = s.get_entropy()
        flags = []
        if s.Characteristics & 0x20000000: flags.append('x')
        if s.Characteristics & 0x40000000: flags.append('w')
        if s.Characteristics & 0x80000000: flags.append('r')
        result['sections'].append({
            'name': name, 'entropy': round(ent,2),
            'flags': ''.join(flags),
            'alert': ent > 7.0 or 'xw' in ''.join(flags)
        })

    # Imports + détection de red flags
    RED_FLAGS = [
        'VirtualAllocEx','WriteProcessMemory','CreateRemoteThread',
        'NtCreateThreadEx','VirtualProtectEx','OpenProcess',
        'CryptUnprotectData','BCryptEncrypt',
        'IsDebuggerPresent','CheckRemoteDebuggerPresent',
    ]
    if hasattr(pe, 'DIRECTORY_ENTRY_IMPORT'):
        for entry in pe.DIRECTORY_ENTRY_IMPORT:
            dll = entry.dll.decode(errors='replace')
            funcs = []
            for imp in entry.imports:
                name = imp.name.decode(errors='replace') if imp.name else f'ord_{imp.or
dinal}'
                funcs.append(name)
                if name in RED_FLAGS:
                    result['suspicious_imports'].append(f'{dll}::{name}')
            result['imports'][dll] = funcs

    return result

def extract_strings_floss(filepath):
    """Utilise FLOSS si disponible, sinon strings basique"""
    try:
```

```
        out = subprocess.check_output(['floss', '--json', filepath],
            timeout=60, stderr=subprocess.DEVNULL)
        data = json.loads(out)
        strings = [s['string'] for s in data.get('floss_stackstrings', [])]
        strings += [s['string'] for s in data.get('floss_decoded_strings', [])]
        return strings
    except Exception:
        # Fallback : strings basiques
        raw = Path(filepath).read_bytes()
        return [m.decode('ascii') for m in
                re.findall(b'[\x20-\x7e]{6,}', raw)]

def find_iocs_in_strings(strings):
    iocs = {'urls': [], 'ips': [], 'domains': [], 'mutexes': [],'registry':[],'paths':[
],'emails':[],'b64':[]
    }
    for s in strings:
        if re.match(r'https?://', s):     iocs['urls'].append(s)
        if re.match(r'[0-9]{1,3}(\.[0-9]{1,3}){3}', s): iocs['ips'].append(s)
        if re.match(r'HKEY_', s):         iocs['registry'].append(s)
        if re.match(r'Global\\\\', s):iocs['mutexes'].append(s)
        if re.match(r'C:\\\\', s):    iocs['paths'].append(s)
        if '@' in s and '.' in s:         iocs['emails'].append(s)
    return {k:list(set(v)) for k,v in iocs.items() if v}

def full_analysis(filepath):
    print(f'Analyse de : {filepath}')
    report = {
        'file': str(filepath),
        'hashes': compute_hashes(filepath),
        'pe': pe_analysis(filepath),
        'strings_iocs': {},
    }
    strings = extract_strings_floss(filepath)
    report['strings_iocs'] = find_iocs_in_strings(strings)
    return report

if __name__ == '__main__':
    report = full_analysis(sys.argv[1])
    print(json.dumps(report, indent=2))
```

CHAPITRE 9

YARA Avancé et Threat Hunting

Écrire des règles qui détectent des familles entières, scanner en mémoire

9.1 YARA en profondeur : modules et conditions complexes

Au volume 1, on a vu les règles YARA basiques. Voici les fonctionnalités avancées qui font la différence entre une règle qui génère des faux positifs et une règle de production.

Le module PE — conditions sur la structure PE

Le module PE de YARA te permet d'écrire des conditions sur la structure PE sans avoir à chercher des bytes spécifiques. Tu peux tester directement l'entropie, les imports, les sections, le timestamp.

YARA avancé — Module PE

```
import "pe"
import "math"

rule AdvancedMalware_Injector_x64 {
    meta:
        description = "Détecte un injecteur x64 avec caractéristiques spécifiques"

    strings:
        $inject_seq = { 48 8B ?? ?? 48 8B ?? ?? FF 15 }  // Appel indirect après déref

    condition:
        // 1. C est bien un PE 64 bits
        pe.machine == pe.MACHINE_AMD64

        // 2. Importation de fonctions d injection
        and pe.imports("kernel32.dll", "VirtualAllocEx")
        and pe.imports("kernel32.dll", "WriteProcessMemory")
        and pe.imports("kernel32.dll", "CreateRemoteThread")

        // 3. Timestamp suspect (1970 ou futur = fakté)
        and (pe.timestamp < 100000 or pe.timestamp > 2000000000)

        // 4. Au moins une section avec entropie elevée
        and for any section in pe.sections : (
            math.entropy(section.raw_data_offset, section.raw_data_size) > 7.0
        )
```

```
        // 5. Ou sequence d assembleur suspecte
        and $inject_seq

        // 6. Pas un gros fichier (installateur, etc)
        and filesize < 5MB
}

// Le module PE permet aussi :
// pe.number_of_sections                  → nombre de sections
// pe.version_info["CompanyName"]          → version info
// pe.dll_characteristics & pe.DLL_CHARACTERISTICS_DYNAMIC_BASE  → ASLR actif
// pe.imphash()                            → IMPHASH pour clustering
// pe.exports("SomeFunction")              → vérifie un export
```

Wildcards, jump et conditions avancées

YARA — Wildcards, sauts et patterns de bytes

```
rule Crypto_AES_Implementation {
    meta:
        description = "Détecte une implémentation directe d AES (pas via BCrypt)"
        note = "Haute entropie des constantes = algorithme embarqué"

    strings:
        // S-Box AES — 16 premiers bytes (signature très fiable)
        $aes_sbox = { 63 7C 77 7B F2 6B 6F C5 30 01 67 2B FE D7 AB 76 }

        // Inverse S-Box (pour déchiffrement)
        $aes_inv_sbox = { 52 09 6A D5 30 36 A5 38 BF 40 A3 9E 81 F3 D7 }

        // Round constants AES
        $aes_rcon = { 01 00 00 00 02 00 00 00 04 00 00 00 08 00 00 00 }

    condition:
        uint16(0) == 0x5A4D  // MZ header
        and ($aes_sbox or $aes_inv_sbox)
        and $aes_rcon

        // Ne pas confondre avec une DLL crypto légitime
        and not pe.imports("bcrypt.dll", "BCryptEncrypt")
}

rule Stealer_AntiAnalysis_Sequence {
    meta:
        description = "Séquence anti-analyse typique des stealers modernes"
```

```
    strings:
        // IsDebuggerPresent suivi de CheckRemoteDebugger
        // Avec wildcard entre les deux (code variable)
        $antidebug_seq = {
            FF 15 ?? ?? ?? ??   // CALL [IsDebuggerPresent]
            [2-20]              // Instructions variables entre les deux
            FF 15 ?? ?? ?? ??   // CALL [CheckRemoteDebuggerPresent]
        }
        // Accès PEB typique du shellcode (PEB Traversal)
        $peb_access = { 65 48 8B 04 25 60 00 00 00 }  // MOV RAX, GS:[0x60]
    condition:
        $antidebug_seq or $peb_access
}
```

9.2 yara-python : intégrer YARA dans tes scripts

yara-python est le binding Python de YARA. Il te permet d'intégrer tes règles dans un pipeline Python, de scanner des répertoires entiers, ou d'analyser des processus en mémoire.

yara-python — Scan fichiers et processus

```
# yara-python — Scanner des processus en mémoire
import yara, psutil, ctypes
from ctypes.wintypes import *

# Compiler les règles
rules = yara.compile(filepath='mes_regles.yar')
# Ou depuis une string
rules = yara.compile(source='''
    rule test {
        strings:
            $s = "evil_string"
        condition:
            $s
    }
''')

# Scanner un fichier
matches = rules.match('suspicious.exe')
for match in matches:
```

```
    print(f'Règle: {match.rule}')
    for string in match.strings:
        print(f'  Match: {string.identifier} à offset {string.instances[0].offset}')

# Scanner un processus en mémoire (Windows, requiert droits admin)
PROCESS_VM_READ = 0x0010
PROCESS_QUERY_INFO = 0x0400
kernel32 = ctypes.windll.kernel32

def scan_process_memory(pid):
    handle = kernel32.OpenProcess(PROCESS_VM_READ | PROCESS_QUERY_INFO, False, pid)
    if not handle:
        return []
    try:
        matches = rules.match(pid=pid)  # yara-python supporte le scan par PID !
        return matches
    finally:
        kernel32.CloseHandle(handle)

# Scanner tous les processus
print('Scanning tous les processus en mémoire...')
for proc in psutil.process_iter(['pid', 'name']):
    try:
        matches = rules.match(pid=proc.info['pid'])
        if matches:
            print(f'MATCH dans {proc.info["name"]} (PID {proc.info["pid"]}) :')
            for m in matches:
                print(f'  → {m.rule}')
    except Exception as e:
        pass  # Accès refusé sur certains processus système
```

9.3 Threat Hunting proactif avec YARA

Le threat hunting consiste à chercher proactivement des compromissions dans ton environnement, avant qu'une alerte se déclenche. Avec YARA et Python, tu peux scanner en continu les nouveaux fichiers dans %TEMP%, ou les processus en mémoire, et alerter immédiatement.

threat_hunter.py — Hunt proactif en temps réel

```
#!/usr/bin/env python3
# threat_hunter.py — Hunt proactif avec YARA
import yara, os, hashlib, sqlite3, time
```

```
from pathlib import Path
from datetime import datetime
import watchdog.observers, watchdog.events

# Compiler toutes les règles du répertoire rules/
RULES_DIR = Path('./yara_rules')
all_rules = {}
for rule_file in RULES_DIR.glob('*.yar'):
    all_rules[rule_file.stem] = str(rule_file)
rules = yara.compile(filepaths=all_rules)
print(f'{len(all_rules)} règles YARA chargées')

# Base de données des fichiers déjà scannés
db = sqlite3.connect('scan_cache.db')
db.execute('CREATE TABLE IF NOT EXISTS scanned (sha256 TEXT PRIMARY KEY, result TEXT)')

def scan_file(filepath):
    filepath = str(filepath)
    try:
        # Calculer le hash pour le cache
        data = Path(filepath).read_bytes()
        if len(data) < 100:  # Skip les fichiers trop petits
            return
        sha256 = hashlib.sha256(data).hexdigest()

        # Vérifier le cache
        cached = db.execute('SELECT result FROM scanned WHERE sha256=?',(sha256,)).fetc
hone()
        if cached:
            return  # Déjà scanné

        # Scanner
        matches = rules.match(filepath, timeout=10)

        if matches:
            alert = {
                'timestamp': datetime.now().isoformat(),
                'file': filepath,
                'sha256': sha256,
                'rules_matched': [m.rule for m in matches]
            }
            print(f'[!] ALERTE YARA : {filepath}')
```

```
            print(f'    SHA256 : {sha256}')
            print(f'    Règles : {[m.rule for m in matches]}')
            # → Envoyer vers SIEM via syslog ou API

        # Mettre en cache
        db.execute('INSERT OR REPLACE INTO scanned VALUES (?,?)',
                   (sha256, str([m.rule for m in matches])))
        db.commit()

    except Exception as e:
        pass  # Fichier verrouillé, accès refusé, etc.

# Watcher sur les répertoires sensibles
class MalwareWatcher(watchdog.events.FileSystemEventHandler):
    def on_created(self, event):
        if not event.is_directory:
            if event.src_path.endswith(('.exe','.dll','.ps1','.bat','.vbs','.hta')):
                print(f'Nouveau fichier détecté : {event.src_path}')
                scan_file(event.src_path)

observer = watchdog.observers.Observer()
watch_dirs = [
    os.environ.get('TEMP', 'C:\\Temp'),
    os.path.expanduser('~/AppData/Roaming'),
    os.path.expanduser('~/AppData/Local/Temp'),
]
handler = MalwareWatcher()
for d in watch_dirs:
    if os.path.exists(d):
        observer.schedule(handler, d, recursive=True)
        print(f'Watching : {d}')

observer.start()
print('Threat Hunter démarré. Ctrl+C pour arrêter.')
try:
    while True: time.sleep(1)
except KeyboardInterrupt:
    observer.stop()
observer.join()
```

CHAPITRE 10

Développer ses Propres Outils RE

Parser le PE, émuler avec Unicorn, hooker des APIs avec Frida, batch Ghidra

10.1 Parser un PE en Python de zéro

Même si pefile fait tout pour toi, comprendre comment parser un PE manuellement te rend meilleur analyste et te permet d'écrire des parseurs pour des cas spéciaux (PE modifiés par un malware, overlay data, sections avec des headers corrompus volontairement).

pe_parser.py — Parser PE de zéro

```
#!/usr/bin/env python3
# pe_parser.py — Parser PE minimal de zéro, sans pefile
import struct
from dataclasses import dataclass
from typing import List

@dataclass
class Section:
    name: str
    virtual_address: int
    virtual_size: int
    raw_offset: int
    raw_size: int
    characteristics: int

class PEParser:
    def __init__(self, filepath):
        with open(filepath, 'rb') as f:
            self.data = f.read()
        self._parse()

    def read(self, offset, size):
        return self.data[offset:offset+size]

    def read_u16(self, offset): return struct.unpack_from('<H', self.data, offset)[0]
    def read_u32(self, offset): return struct.unpack_from('<I', self.data, offset)[0]
    def read_u64(self, offset): return struct.unpack_from('<Q', self.data, offset)[0]

    def _parse(self):
```

```
        # DOS Header
        if self.read(0, 2) != b'MZ':
            raise ValueError('Pas un fichier PE (pas de MZ header)')

        # e_lfanew = offset vers PE Header
        self.pe_offset = self.read_u32(0x3C)

        # PE Signature
        if self.read(self.pe_offset, 4) != b'PE\x00\x00':
            raise ValueError('PE signature invalide')

        # COFF File Header (20 bytes après PE sig)
        coff = self.pe_offset + 4
        self.machine         = self.read_u16(coff + 0)
        self.num_sections    = self.read_u16(coff + 2)
        self.timestamp       = self.read_u32(coff + 4)
        self.opt_header_size = self.read_u16(coff + 16)
        self.characteristics = self.read_u16(coff + 18)

        self.is_64bit = (self.machine == 0x8664)

        # Optional Header
        opt = coff + 20
        self.magic = self.read_u16(opt)
        if self.magic == 0x20B:  # PE32+
            self.imagebase  = self.read_u64(opt + 24)
            self.entrypoint = self.read_u32(opt + 16)
        else:  # PE32
            self.imagebase  = self.read_u32(opt + 28)
            self.entrypoint = self.read_u32(opt + 16)

        # Section Table
        section_offset = opt + self.opt_header_size
        self.sections: List[Section] = []
        for i in range(self.num_sections):
            s_off = section_offset + i * 40  # Chaque section header = 40 bytes
            name = self.data[s_off:s_off+8].rstrip(b'\x00').decode('ascii','replace')
            self.sections.append(Section(
                name            = name,
                virtual_size    = self.read_u32(s_off + 8),
                virtual_address = self.read_u32(s_off + 12),
                raw_size        = self.read_u32(s_off + 16),
```

```
                raw_offset      = self.read_u32(s_off + 20),
                characteristics = self.read_u32(s_off + 36),
            ))

    def rva_to_offset(self, rva):
        """Convertir une RVA (adresse relative) en offset fichier"""
        for s in self.sections:
            if s.virtual_address <= rva < s.virtual_address + s.virtual_size:
                return s.raw_offset + (rva - s.virtual_address)
        return None

    def report(self):
        arch = 'x64' if self.is_64bit else 'x86'
        print(f'Architecture  : {arch}')
        print(f'Entry Point   : 0x{self.entrypoint:08X} (RVA)')
        print(f'Image Base    : 0x{self.imagebase:X}')
        print(f'Timestamp     : {self.timestamp}')
        print(f'Sections ({self.num_sections}) :')
        for s in self.sections:
            rwx = ''
            if s.characteristics & 0x40000000: rwx += 'r'
            if s.characteristics & 0x80000000: rwx += 'w'
            if s.characteristics & 0x20000000: rwx += 'x'
            print(f'  {s.name:10} VA:0x{s.virtual_address:08X}  Size:0x{s.virtual_size:
06X}  [{rwx}]')

if __name__ == '__main__':
    import sys
    pe = PEParser(sys.argv[1])
    pe.report()
```

10.2 Hooker des APIs avec Frida

Frida est un framework de dynamic instrumentation. Il te permet d'injecter du JavaScript dans n'importe quel processus en cours d'exécution, de hooker des fonctions, de lire et modifier de la mémoire, et de tracer les appels. C'est plus puissant et plus flexible qu'x64dbg pour certains cas.

L'avantage de Frida sur x64dbg pour l'analyse malware : tu peux écrire des hooks complexes en JavaScript avec toute la puissance d'un vrai langage, tu peux hooker plusieurs processus simultanément, et tu peux modifier les arguments et retours de fonctions en temps réel de façon beaucoup plus flexible que les scripts x64dbg.

Frida — Hooks JavaScript sur APIs Windows

```
# Frida — Hooker toutes les APIs de création de processus
# pip install frida-tools --break-system-packages
# Télécharger frida-server depuis https://github.com/frida/frida/releases

# Script Python pour lancer le hook
import frida, sys

# Script JavaScript injecté dans le processus cible
JSCRIPT = """
'use strict';

// Hook CreateProcessW
const createProcessW = Module.findExportByName('kernel32.dll', 'CreateProcessW');
if (createProcessW) {
    Interceptor.attach(createProcessW, {
        onEnter: function(args) {
            // args[0] = lpApplicationName (wchar_t*)
            // args[1] = lpCommandLine (wchar_t*)
            const appName  = args[0].isNull() ? 'NULL' : args[0].readUtf16String();
            const cmdLine  = args[1].isNull() ? 'NULL' : args[1].readUtf16String();
            // args[6] = dwCreationFlags
            const flags    = args[6].toInt32();
            const suspended = (flags & 0x4) !== 0;

            console.log('[CreateProcessW]');
            console.log('  App  : ' + appName);
            console.log('  Cmd  : ' + cmdLine);
            console.log('  Suspended: ' + suspended + (suspended ? ' ← PROCESS HOLLOWI
NG ?' : ''));
        }
    });
}

// Hook VirtualAllocEx
const virtualAllocEx = Module.findExportByName('kernel32.dll', 'VirtualAllocEx');
if (virtualAllocEx) {
    Interceptor.attach(virtualAllocEx, {
        onEnter: function(args) {
            const protect = args[4].toInt32();
            const sizeVal = args[2].toInt32();
```

```
            if (protect === 0x40) {  // PAGE_EXECUTE_READWRITE
                console.log('[VirtualAllocEx] PAGE_EXECUTE_READWRITE !');
                console.log('  Size: 0x' + sizeVal.toString(16));
                console.log('  → Probable injection de shellcode');
                console.log(Thread.backtrace().map(DebugSymbol.fromAddress).join('\n'))
;
            }
        }
    });
}
"""

# Attacher à un processus par nom
def on_message(message, data):
    if message['type'] == 'send':
        print('[Frida]', message['payload'])

target_process = sys.argv[1] if len(sys.argv) > 1 else 'malware.exe'
session = frida.attach(target_process)
script = session.create_script(JSCRIPT)
script.on('message', on_message)
script.load()
print(f'[*] Hook Frida actif sur {target_process}. Ctrl+C pour stopper.')
sys.stdin.read()
```

10.3 Automatiser Ghidra en batch (Headless)

Ghidra peut tourner sans interface graphique en mode 'headless'. C'est parfait pour analyser des dizaines de samples automatiquement et extraire des informations (imports, strings, fonctions) sans ouvrir l'UI.

Ghidra Headless — Analyse batch automatique

```
# Ghidra Headless — Analyser un fichier et extraire les infos

# Depuis la ligne de commande :
# analyzeHeadless /path/to/project MyProject \
#     -import malware.exe \
#     -postScript ExportImports.java output.json \
#     -scriptPath /path/to/scripts/

# Script Java pour l'export (ExportImports.java)
```

```
# import ghidra.app.script.GhidraScript;
# import com.google.gson.Gson;
# ...
# public class ExportImports extends GhidraScript {
#     public void run() throws Exception {
#         Map<String,List<String>> imports = new HashMap<>();
#         if (currentProgram.hasImportedSymbols()) {
#             for (ExternalLocation loc : ...) {
#                 // Collecter imports
#             }
#         }
#         // Écrire en JSON
#         Gson gson = new Gson();
#         Files.write(Paths.get(getScriptArgs()[0]),
#                     gson.toJson(imports).getBytes());
#         println('Export terminé');
#     }
# }

# Script Python pour lancer analyzeHeadless sur un répertoire
import subprocess, os, json
from pathlib import Path

GHIDRA_HOME = '/opt/ghidra'
ANALYZE     = f'{GHIDRA_HOME}/support/analyzeHeadless'
PROJECT_DIR = '/tmp/ghidra_projects'
SCRIPT_DIR  = './ghidra_scripts'

def analyze_batch(sample_dir):
    results = []
    for exe_file in Path(sample_dir).glob('*.exe'):
        project_name = exe_file.stem
        output_file  = f'/tmp/{project_name}_iocs.json'

        cmd = [
            ANALYZE, PROJECT_DIR, project_name,
            '-import', str(exe_file),
            '-postScript', 'ExtractIOCs.py', output_file,
            '-scriptPath', SCRIPT_DIR,
            '-noanalysis' if os.path.exists(output_file) else '-analyse',  # Skip si dé
jà fait
```

```
            '-deleteProject',  # Supprimer le projet après (économie disque)
        ]

        subprocess.run(cmd, capture_output=True, timeout=300)

        if os.path.exists(output_file):
            with open(output_file) as f:
                results.append({'file': str(exe_file), **json.load(f)})
        else:
            results.append({'file': str(exe_file), 'error': 'Analysis failed'})

    return results

# Analyser un répertoire entier de samples
all_results = analyze_batch('./samples')
with open('batch_analysis.json', 'w') as f:
    json.dump(all_results, f, indent=2)
print(f'{len(all_results)} samples analysés')
```

Ce que tu maîtrises maintenant

Après ces deux volumes, tu as les outils pour analyser la grande majorité des malwares que tu rencontreras dans ton SOC. Du sample .NET obfusqué avec ConfuserEx au shellcode CobaltStrike Beacon, du dropper packé avec UPX au RAT C++ avec anti-debug — tu sais comment approcher chaque cas.

Ce qui fera la différence dans ton évolution : la régularité de la pratique. Analyse 2-3 samples par semaine. Varie les familles. Documente tes analyses. Compare avec les rapports publics sur ANY.RUN ou Hybrid Analysis. Participe au FLARE-ON Challenge de Mandiant chaque automne — c'est le meilleur benchmark pour mesurer ta progression.

La communauté RE est ouverte et documente énormément ses recherches. Suis les blogs de : hasherezade, Didier Stevens, OALabs, Hexacorn, The DFIR Report, MalwareTech, Kaspersky GReAT, Mandiant FLARE. Chaque article publié est une analyse gratuite dont tu peux t'inspirer.

Volume 1

→ Fondamentaux : assembleur, PE, Ghidra, x64dbg, analyse statique et dynamique, familles de malwares, rapport SOC

Volume 2

→ Approfondissement : Ghidra scripting, .NET RE, shellcode, crypto, VM protectors, fileless, automatisation, YARA avancé, développement d'outils

Étape suivante

→ Spécialisation : APT analysis, kernel RE (drivers), mobile RE (Android), exploit analysis, CTF specialisation

Un bon analyste RE n'est pas celui qui n'a jamais bloqué — c'est celui qui a appris à débloquer.